

NYC TAXI DURATION

Production ML Engineering Guide

Basit bir regression modelinden güvenilir, izlenebilir ve geri alınabilir bir production sistemine

Kapsam	Uçtan uca
Problem	NYC yellow taxi yolculuk süresi tahmini
Ana vurgu	Data contract, CI/CD, deployment, monitoring, retraining, rollback
Model	Median baseline + HistGradientBoostingRegressor
Yazar	Ugur Alkan
Sürüm	1.2.3 September 2026

Bu doküman repodaki gerçek kodla birebir eşleşir. Sentetik doğrulama metrikleri yalnızca sistem smoke testi içindir; gerçek TLC model performansı değildir.

Bu materyal nasıl çalışılmalı

Belge dört katmanlı okunabilir. Önce problem ve leakage sınırını anlayın. Sonra offline training akışını ve release gate mantığını izleyin. Ardından online API, container ve deployment katmanlarına geçin. Son bölümdeki tam kaynak kod ekini açık bir editörle birlikte okuyun ve laboratuvar komutlarını çalıştırın.

- Birinci geçiş: Bölüm 1-5 ile sistemin neden böyle tasarlandığını öğrenin.
- İkinci geçiş: Bölüm 6-12 ile kodun bileşenlerini ve çalışma sırasını izleyin.
- Üçüncü geçiş: Laboratuvarları çalıştırın, bilerek gate kırın ve rollback senaryosunu inceleyin.
- Referans kullanımı: Dosya kataloğu ve kaynak kod eki üzerinden belirli bir modüle dönün.

Önemli ayrım: production-ready bir repository ile şirketinizin production platformuna deploy edilmiş sistem aynı şey değildir. Bu repo uygulama kodunu ve platform sözleşmesini sağlar; gerçek cluster kimliği, TLS, secret manager, kalıcı telemetry, SLO alert'leri ve yönetim organizasyona bağlanmalıdır.

İçindekiler

Production ML Engineering Guide	1
Bu materyal nasıl çalışılmalı	2
İçindekiler	3
1. Problem tanımı ve başarı ölçütü	7
1.1 Neden bu bir production problemi?	7
1.2 Kabul kriterleri	7
2. Dataset, source contract ve leakage	7
2.1 Feature availability matrix	7
2.2 Quality filtreleri	8
3. Sistem mimarisi	8
3.1 Boundary'ler ve sorumluluklar	8
4. Repository yapısı ve dosya kataloğu	9
5. Configuration sistemi	9
6. Ingestion ve veri doğrulama	10
6.1 Publication-aware dönem çözümleme	10
6.2 Atomik download	10
6.3 SHA-256 lineage	10
6.4 Validation ve rolling merge sırası	10
7. Feature engineering ve training-serving skew	10
8. Model, target dönüşümü ve baselines	11
8.1 Median baseline	11
8.2 HistGradientBoostingRegressor	11
8.3 Log target	11
9. Temporal split, evaluation ve release gates	11
9.1 Gate mantığı	12
9.2 Champion confidence karşılaştırması	12
9.3 Doğrulanmış sentetik smoke sonucu	12
10. Artifact, metadata ve reproducibility	12
11. FastAPI serving katmanı	13
11.1 Lifespan ve readiness	13
11.2 Request contract	13
11.3 Endpoint'ler	13
11.4 Observability	13
12. Container ve local observability stack	14
13. CI: code quality ve supply-chain kontrolleri	14
14. CD: gated retraining, staging, promotion, rollback	14

15. Kubernetes deployment modeli	15
15.1 Eksik platform bileşenleri	15
16. Monitoring: operasyon, drift ve delayed labels	15
16.1 Üç monitoring katmanı	15
16.2 PSI	15
16.3 Neden prediction drift yetmez?	15
17. Test stratejisi	16
18. Local çalışma laboratuvarı	16
18.1 Kurulum	16
18.2 Sentetik smoke data	16
18.3 API	16
18.4 Resmi data	16
18.5 Drift	16
19. Failure injection çalışmaları	17
20. Security ve threat model	17
21. Production'a geçmeden önce checklist	17
22. v1.2 sonrası teknik yol haritası	18
23. Kaynaklar	18
Ek A. Tam kaynak kod referansı	19
A.1 .env.example	19
A.2 .github/workflows/ci.yml	19
A.3 .github/workflows/retrain.yml	19
A.4 DATA_CARD.md	21
A.5 Dockerfile	22
A.6 Makefile	22
A.7 configs/base.yaml	23
A.8 configs/development.yaml	23
A.9 configs/production.yaml	24
A.10 configs/staging.yaml	24
A.11 docker-compose.yml	24
A.12 docs/adr/0001-problem-and-feature-contract.md	24
A.13 docs/adr/0002-time-split.md	24
A.14 docs/adr/0003-simple-model.md	25
A.15 docs/adr/0004-zone-encoding.md	25
A.16 docs/adr/0005-release-comparison-and-runtime.md	25
A.17 docs/adr/0006-v12-release-reliability.md	26
A.18 docs/adr/0007-champion-noninferiority.md	26
A.19 docs/model-card.md	27
A.20 docs/runbook.md	27

A.21 k8s/base/deployment.yaml	28
A.22 k8s/base/kustomization.yaml	28
A.23 k8s/base/service.yaml	28
A.24 k8s/overlays/production/kustomization.yaml	28
A.25 k8s/overlays/staging/kustomization.yaml	29
A.26 monitoring/grafana/provisioning/dashboards/provider.yaml	29
A.27 monitoring/grafana/provisioning/datasources/prometheus.yaml	29
A.28 monitoring/prometheus.yaml	29
A.29 pyproject.toml	29
A.30 scripts/generate_sample_data.py	30
A.31 scripts/smoke_test.sh	31
A.32 src/taxi_duration/__init__.py	31
A.33 src/taxi_duration/cli.py	31
A.34 src/taxi_duration/config.py	33
A.35 src/taxi_duration/data/__init__.py	34
A.36 src/taxi_duration/data/ingest.py	34
A.37 src/taxi_duration/data/validation.py	36
A.38 src/taxi_duration/features/__init__.py	37
A.39 src/taxi_duration/features/builder.py	37
A.40 src/taxi_duration/logging.py	38
A.41 src/taxi_duration/models/__init__.py	38
A.42 src/taxi_duration/models/artifact.py	38
A.43 src/taxi_duration/models/factory.py	40
A.44 src/taxi_duration/models/predict.py	40
A.45 src/taxi_duration/monitoring/__init__.py	41
A.46 src/taxi_duration/monitoring/drift.py	41
A.47 src/taxi_duration/monitoring/profile.py	42
A.48 src/taxi_duration/serving/__init__.py	43
A.49 src/taxi_duration/serving/app.py	43
A.50 src/taxi_duration/serving/metrics.py	45
A.51 src/taxi_duration/serving/schemas.py	45
A.52 src/taxi_duration/training/__init__.py	46
A.53 src/taxi_duration/training/evaluate.py	46
A.54 src/taxi_duration/training/gates.py	46
A.55 src/taxi_duration/training/pipeline.py	48
A.56 src/taxi_duration/training/split.py	51
A.57 tests/conftest.py	51
A.58 tests/contract/test_api_schema.py	52
A.59 tests/integration/test_api.py	52
A.60 tests/integration/test_training_pipeline.py	53

A.61 tests/unit/test_config.py 54

A.62 tests/unit/test_drift.py 55

A.63 tests/unit/test_features.py 55

A.64 tests/unit/test_gates.py 56

A.65 tests/unit/test_ingest.py 57

A.66 tests/unit/test_monitoring.py 58

A.67 tests/unit/test_period.py 58

A.68 tests/unit/test_predict.py 59

A.69 tests/unit/test_validation.py 59

1. Problem tanımı ve başarı ölçütü

İş problemi şudur: Bir yolcu pickup zamanını, pickup taxi zone'unu, gitmek istediği drop-off zone'unu ve yolcu sayısını verdiğinde, yolculuk başlamadan önce beklenen süreyi dakika cinsinden tahmin etmek. Tahmin bir point estimate'tir; kullanıcıya belirsizlik aralığı sunulmaz. Bu, sürüm 1'in bilinçli sınırıdır.

Matematiksel olarak model $f(X) = y_{\text{hat}}$ kurar. X yalnızca karar anında bilinen değişkenlerden oluşur. Ground truth y , drop-off tamamlandıktan sonra pickup ve drop-off timestamp farkından elde edilir. Böylece target gelecekte oluşur fakat feature set geleceğe bakmaz.

```
duration_minutes = (tpep_dropoff_datetime - tpep_pickup_datetime).total_seconds() / 60
```

1.1 Neden bu bir production problemi?

TLC verisi aylık yayımlanır, şema değişebilir, gecikmeli label üretir ve mevsimsel/operasyonel drift taşır. Dolayısıyla ingestion, veri kalitesi, yeniden eğitim ve promotion doğal bir lifecycle oluşturur. NYC TLC veri sayfası trip kayıtlarının aylık ve tipik olarak yaklaşık iki aylık gecikmeyle yayımlandığını, Parquet formatının kullanıldığını ve küçük şema değişiklikleri olabileceğini açıkça belirtir.

1.2 Kabul kriterleri

Boyut	Kabul koşulu
Fonksiyonel	Geçerli request için dakika tahmini ve model/contract sürümü döner.
Veri	Eksik kolon veya tümü geçersiz batch deterministik biçimde hata verir.
ML	Candidate, median baseline'ı ayarlı marjla geçer ve tüm absolute threshold'ları sağlar.
Operasyon	Liveness, readiness, structured logs ve Prometheus metric'leri çalışır.
Release	Aynı image digest staging smoke testinden production'a promote edilir.
Recovery	Başarısız production rollout önceki ReplicaSet'e geri alınır.

2. Dataset, source contract ve leakage

Kaynak resmi NYC TLC yellow taxi trip record Parquet dosyalarıdır. TLC; pickup/drop-off zamanları ve lokasyonları, trip distance, itemized fare, rate, payment type ve driver-reported passenger count gibi alanlar bulunduğunu, kaydın teknoloji sağlayıcılarından geldiğini ve doğruluğu garanti etmediğini söyler. Bu uyarı veri kalitesi tasarımının neden zorunlu olduğunu açıklar.

2.1 Feature availability matrix

Alan	Feature?	Gerekçe
tpep_pickup_datetime	Evet	Request anında bilinir; saat ve weekday türetilir.
PULocationID	Evet	Pickup zone request anında bilinir.
DOLocationID	Evet	Kullanıcının hedefi olarak bilinir.
passenger_count	Evet	Pickup sırasında bilinir; 1-6 contract aralığı.
tpep_dropoff_datetime	Hayır	Sadece label üretir; gelecekte oluşur.
trip_distance	Hayır	Gerçekleşmiş taximeter mesafesi trip sonunda kesinleşir.
fare/tip/tolls/total	Hayır	Post-trip ve target-adjacent leakage.
payment_type	Hayır	Çoğu senaryoda trip sonunda kesinleşir.

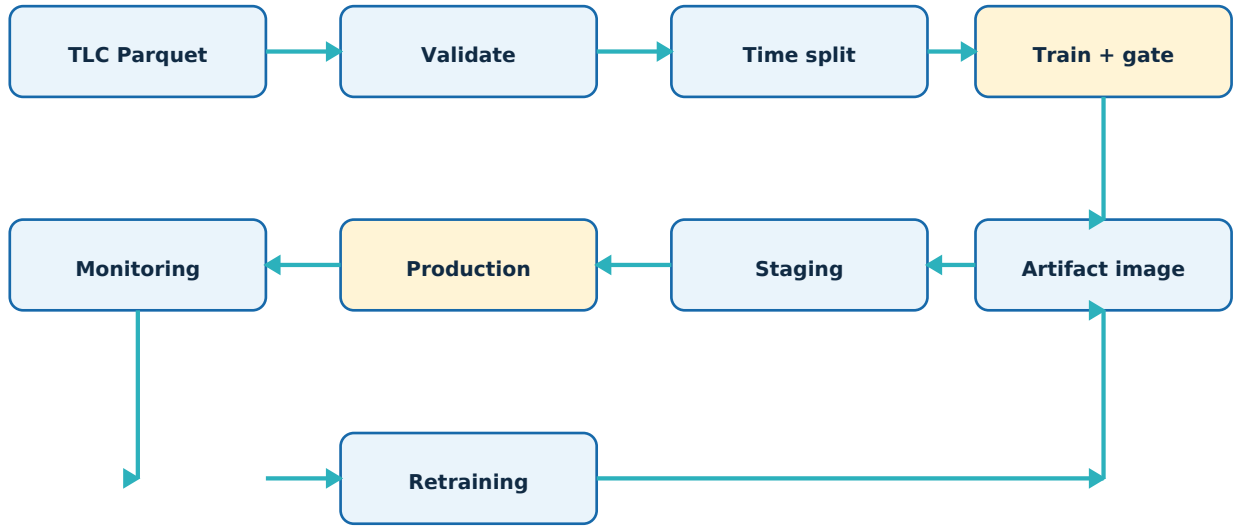
Bu tablo modelden önce gelir. Feature importance'a bakıp sızıntı keşfetmek geç bir kontroldür; asıl kontrol decision-time data contract'tır. `INFERENCE\_COLUMNS` allowlist'i bu kararı kod içinde enforce

eder.

2.2 Quality filtreleri

- Beş zorunlu kaynak kolonu yoksa batch hemen reddedilir.
- Timestamp parse edilemeyen satırlar elenir.
- Duration 1-120 dakika dışında ise outlier/bozuk kayıt kabul edilmez.
- Passenger count 1-6 ve routable zone ID 1-263 arasında olmalıdır.
- Pickup source dosyasının ayı dışında ise satır elenir ve ayrıca sayılır.
- Beş kaynak kolonu tamamen aynı olan duplicate satırların ilki tutulur.
- Temiz veri pickup time'a göre sıralanır; boş kalan batch hata verir.
- Rejection, out-of-period, duplicate oranları ve valid volume gate edilir.

3. Sistem mimarisi



A failed gate or smoke test never advances the candidate

Sistem iki ana döngü taşır. Offline döngü source data'yı model artifact'ına dönüştürür. Online döngü artifact'ı request başına çalıştırır ve telemetry üretir. Monitoring bu iki döngüyü bağlar: labelsiz input/prediction drift erken sinyal, gecikmeli gerçek duration ise performans sinyalidir.

3.1 Boundary'ler ve sorumluluklar

Boundary	Girdi	Çıktı	Sahip olduğu garanti
Ingestion	year/month	immutable Parquet + SHA	atomik download, tekrar kullanılabilir checksum
Validation	raw DataFrame	clean DataFrame + report	schema/domain kuralları
Training	clean temporal data	fitted Pipeline	train/serve transform birliği
Gate	metrics + thresholds	pass/fail + reasons	kötü candidate ilerlemez
Artifact	model + lineage	joblib + JSON	kod/veri/config izlenebilirliği
Serving	strict JSON	versioned prediction	timezone, bounds, availability
Deployment	image digest	running ReplicaSet	same-bit promotion + rollback

4. Repository yapısı ve dosya kataloğu

Notebook merkezli tasarım yoktur. Eğitim, monitoring ve serving import edilebilir modüller ve CLI entrypoint'leri olarak tanımlıdır. Notebook ancak keşif için eklenebilir; release path notebook çalıştırmaz.

Konum	Görev
configs/	Base değerler ve development/staging/production override'ları.
src/taxi_duration/data	HTTP ingestion, checksum, schema ve domain validation.
src/taxi_duration/features	Tek leakage-safe feature transformer.
src/taxi_duration/models	Estimator factory, artifact metadata ve güvenilir load/save.
src/taxi_duration/training	Chronological split, metrics, gates ve orchestration.
src/taxi_duration/serving	Pydantic contract, FastAPI lifecycle, metrics ve endpoints.
src/taxi_duration/monitoring	Reference profile ve PSI drift hesapları.
tests/	Unit, contract ve component integration testleri.
.github/workflows	Code CI ve gated retrain-to-production workflow.
k8s/	Deployment/service base ve environment overlay'leri.
monitoring/	Prometheus scrape ve Grafana provisioning.
docs/	Architecture decisions, model card ve runbook.

5. Configuration sistemi

YAML config dosyaları `AppConfig` ve alt Pydantic modelleriyle `extra=forbid` şeklinde doğrulanır. Yanlış yazılmış bir key sessizce ignore edilmez. Development ve production dosyaları `extends: base.yaml` ile yalnızca farkları belirtir. `\_deep\_merge` nested dictionary'leri korur; scalar veya liste override'ı doğrudan değiştirir.

Config grubu	Kontrol ettiği davranış
data	dizinler, URL, sınırlar, rolling window ve source lookback
split	train ve validation fraction; kalan otomatik test
model	iteration, learning rate, leaves, L2, leaf minimum
gates	quality, MAE, tail, baseline, stability ve champion confidence
serving	bind adresi, port ve ortak output clip bounds
monitoring	numeric/categorical/prediction PSI ve unseen-route sınırı

Production threshold'ları veri gördükten sonra keyfi biçimde ayarlanmamalıdır. Önceden tanımlı risk toleransından türetilmeli, code review ile değişmeli ve historical backtest ile kalibre edilmelidir.

6. Ingestion ve veri doğrulama

6.1 Publication-aware dönem çözümü

Takvimde iki ay geriye gitmek dosyanın yayımlandığını kanıtlamaz. `resolve-period` önce bu adayı HEAD ile yoklar, bulunmazsa `source\_lookback\_months` sınırında ay ay geriye gider. Bulduğu son ayı üç aylık rolling window'un sonu yapar. Hiçbir nesne doğrulanamazsa workflow fail closed davranır; yanlış aya boş/404 download denemez. Yalnızca 200/206 'yayımlanmış', yalnızca 403/404 'yayımlanmamış' sayılır; ağ hatası veya beklenmeyen bir status üç denemeden sonra hata verir, böylece geçici bir kesinti eğitim penceresini sessizce eski bir aya kaydıramaz.

```
taxi-ml resolve-period --config-path configs/staging.yaml
# year=2026
# month=07
# period=2026-07
# window=2026-05,2026-06,2026-07
```

6.2 Atomik download

`download\_month` doğrudan final dosyaya yazmaz. Önce `.part` uzantılı geçici dosyaya stream eder, HTTP status'ü doğrular ve başarılı tamamlanınca `replace` ile atomik olarak final isme taşır. Exception olursa partial dosya silinir. Mevcut final dosya overwrite istenmedikçe yeniden indirilmez.

```
with httpx.stream('GET', url, follow_redirects=True, timeout=120.0) as response:
    response.raise_for_status()
    with temporary.open('wb') as output:
        for chunk in response.iter_bytes(chunk_size=1024 * 1024):
            output.write(chunk)
    temporary.replace(destination)
```

6.3 SHA-256 lineage

Her rolling-window dosyasının checksum'u metrics manifestine yazılır; sıralı manifestin hash'i model metadata'ya bağlanır. Böylece üç aylık training input tek bir lineage kimliğiyle izlenirken dosya bazında kanıt da korunur.

6.4 Validation ve rolling merge sırası

- Önce kolon varlığı doğrulanır; aksi durumda downstream KeyError yerine açıklayıcı contract error üretilir.
- Sonra timestamp coercion ve target construction yapılır.
- Her dosya kendi source ayına göre doğrulanır; cross-month stray satırlar elenir.
- Exact duplicate mask ve bütün domain koşulları birlikte uygulanır.
- Geçerli aylık frame'ler birleştirilir, pickup time'a göre stable sort edilir.
- Rapor oranı batch tamamen boş olsa dahi division-by-zero üretmez.

7. Feature engineering ve training-serving skew

`TripFeatureBuilder` scikit-learn estimator contract'ını uygular. `fit` öğrenilecek istatistik olmadığı için yalnızca input feature isimlerini kaydeder. `transform` allowlist kolonlarını doğrular, timestamp'i parse eder ve DataFrame üretir. Transformer fitted model Pipeline'ının içindedir; API bağımsız bir feature script çalıştırmaz.

Feature	Formül/mantık	Neden
route_target	TargetEncoder(PU*1000+DO)	Route log-süre ortalaması (simetrikse $\approx$ median)
pickup_zone_target	TargetEncoder(PULocationID)	Sparse route fallback
dropoff_zone_target	TargetEncoder(DOLocationID)	Hedef zone etkisi

Feature	Formül/mantık	Neden
passenger_count	numeric cast	Basit kapasite/operasyon etkisi
hour_sin/cos	$\sin/\cos(2\pi \cdot \text{hour}/24)$	23:59 ile 00:01 yakınlığını korur
weekday_sin/cos	$\sin/\cos(2\pi \cdot \text{weekday}/7)$	Haftanın çevrimsel yapısı
is_weekend	$\text{weekday} \geq 5$	Rejim ayrımı
is_rush_hour	07-10 veya 16-20	Basit trafik rejimi proxy'si

TLC zone ID sırası coğrafi değildir. v1.2 route, pickup ve drop-off kategorilerini cross-fitted TargetEncoder ile kodlar: training satırı kendi label'ını görmez, rare/unseen route global/zone ortalamasına shrink olur. `ordinal` yalnızca karşılaştırma koludur.

8. Model, target dönüşümü ve baselines

8.1 Median baseline

DummyRegressor(strategy='median') zorunlu referanstır. Candidate'ın kabul edilmesi yalnızca absolute MAE threshold'una bağlı değildir; median tahmine göre yeterli relative improvement da gerekir. Baseline olmadan 8 dakikalık MAE'nin anlamlı mı zayıf mı olduğu anlaşılamaz.

8.2 HistGradientBoostingRegressor

Histogram-based gradient boosting, tabular nonlinear ilişkileri düşük operasyonel karmaşıklıkla öğrenir. Loss `absolute\_error` modelin conditional median'a yaklaşmasını sağlar. Built-in random early stopping kapalıdır; staged prediction eğrisi kronolojik validation parçasında değerlendirilir ve en iyi iteration yeniden fit edilir.

8.3 Log target

`TransformedTargetRegressor` training sırasında $y' = \log(1+y)$ uygular, prediction'ı $\exp(y')-1$ ile dakika ölçeğine döndürür. Sağ kuyruk sıkıştırılır. `check\_inverse=False`, sayısal inverse kontrol overhead'ini kaldırır; iki fonksiyon analitik olarak eşleştirilmiştir.

```
Pipeline([
    ('features', TripFeatureBuilder()),
    ('regressor', TransformedTargetRegressor(
        regressor=HistGradientBoostingRegressor(loss='absolute_error', ...),
        func=np.log1p, inverse_func=np.expm1, check_inverse=False,
    )),
])
```

9. Temporal split, evaluation ve release gates

Üç ayrı aylık dosya önce kendi ay contract'ıyla temizlenir ve birleşir. Pickup zamanına göre ilk %70 train, sonraki %15 validation, son %15 test yapılır. Böylece final tail daha sonraki zamanı temsil eder. `min\_valid\_rows` küçük veya eksik batch'in metrik şansıyla promote edilmesini engeller.

Metric	Definition	Interpretation
MAE	$\text{mean}(y - y_{\text{hat}})$	Typical absolute error in minutes; directly readable.
RMSE	$\sqrt{\text{mean}((y - y_{\text{hat}})^2)}$	Penalizes large misses more strongly than MAE.
R2	$1 - \text{SSE}/\text{SST}$	Variance explained relative to a mean predictor; can be negative.
p95 AE	95th percentile of $ \text{error} $	Tail reliability: 95% of errors fall below this value.
Bias	$\text{mean}(y_{\text{hat}} - y)$	Positive means systematic overprediction; negative underprediction.
PSI	$\text{sum}((a-e) \ln(a/e))$	Distribution shift heuristic, not a causal failure diagnosis.

9.1 Gate mantığı

- Validation MAE ayarlı maximum'u aşmamalı.
- Final test MAE ayrı maximum'u aşmamalı.
- Test p95 absolute error tail-risk sınırında kalmalı.
- Validation MAE / baseline MAE - 1, gerekli relative improvement'tan kötü olmamalı.
- Validation ve test MAE farkı stability sınırını aşmamalı.
- Rejection, out-of-period ve duplicate oranları sınırı aşmamalı; valid rows minimum'u sağlamalı.
- Candidate champion'a göre non-inferior olmalı: paired absolute-error farkının saat-cluster'lı tek taraflı upper bound'u pozitif marjı (champion MAE'nin %1'i) aşmamalı.

Gate fail olduğunda metrics JSON yine yazılır, fakat model artifact'ı paketlenmez ve command non-zero exit verir. Bu sıra tanılama kanıtını korur, zayıf candidate'ın deploy edilmesini engeller.

9.2 Champion confidence karşılaştırması

Candidate ve production champion aynı test satırlarında skorlanır. Her satır için $d = |y_{\text{candidate}}| - |y_{\text{champion}}|$ alınır (pozitif = candidate daha kötü). Karar kuralı non-inferiority'dir: d ortalaması + $z \cdot SE$, $m = \text{champion MAE} \times \text{max_regression_vs_champion}$ marjını aşarsa promotion bloklanır. SE, aynı saatteki yolculuklar aynı trafiği paylaştığı için pickup saati üzerinden cluster-robust hesaplanır. Marj sıfır olsaydı kural bir üstünlük testine dönüşürdü: champion kadar iyi bir retrain yaklaşık %95 olasılıkla bloklanırdı. Bu yüzden config marjın pozitif olmasını zorunlu tutar (ADR 0007).

9.3 Doğrulanmış sentetik smoke sonucu

Kanıt	Sonuç
Rolling input	3 x 1,200 = 3,600 geçerli sentetik satır
Split	2,520 train / 540 validation / 540 test
Validation MAE	8.07 dakika
Test MAE	7.38 dakika; tüm development gate'leri geçti
Sonraki ay MAE	9.03 dakika (1,200 satır)
Drift	Yeni ayda unseen route %26.67; critical alarm üretildi

Bu sayılar plumbing, gate ve alarm davranışını doğrular; gerçek TLC performansı değildir. Gerçek threshold kalibrasyonu `docs/benchmark-protocol.md` prosedürüyle ayrıca yapılmalıdır.

10. Artifact, metadata ve reproducibility

Artifact	İçerik	Amaç
model.joblib	Feature transformer + fitted regressor	Tek train/serve execution object
metadata.json	version, UTC, Git SHA, hashes, metrics, contract	Lineage ve API görünürlüğü
metrics.json	quality, rows, periods, baseline, candidate, gate	Review ve audit evidence
reference_profile.json	numeric/categorical/prediction dağılımları	Drift reference

Joblib pickle temellidir ve untrusted input için güvenli değildir. Model yalnızca CI/retraining tarafından üretilen trusted artifact'tan yüklenmelidir. Daha sıkı ortamlarda imza/attestation, artifact registry access policy ve compatibility manifest eklenmelidir.

Config SHA, `extends` sonrası tamamen çözülmüş canonical JSON'dan üretilir; base.yaml değişikliği override dosyası aynı kalsa da hash'i değiştirir. Runtime/dev lock'ları transitive package ve wheel hash'lerini taşır. Loader Python minor, scikit-learn, NumPy, pandas ve joblib uyumsuzluğunu reddeder.

11. FastAPI serving katmanı

Online inference path



The fitted Pipeline owns feature transformation and prediction

11.1 Lifespan ve readiness

App startup'ta config, model ve metadata'yı bir kez yükler. Başarırsa `ready=True`; artifact yoksa process yine yaşayabilir fakat readiness 503 döner. Bu ayırım Kubernetes'in broken pod'a trafik göndermemesini, buna rağmen process diagnostic endpoint'inin erişilebilir kalmasını sağlar.

11.2 Request contract

- Unknown fields forbidden: istemeden fare_amount gibi leakage input'u kabul edilmez.
- Routable zone 1-263, passenger 1-6 aralığında olmalıdır.
- Pickup datetime UTC offset içermelidir; naive datetime reddedilir.
- Instant America/New_York local time'a çevrilir ve source training semantiğiyle eşlenir.
- Response prediction ile model_version ve feature_contract_version döndürür.
- Prediction, training/evaluation ile aynı `predict\_duration` bounded helper'ından geçer.

11.3 Endpoint'ler

Endpoint	Amaç	Failure semantics
GET /health/live	Process yaşıyor mu?	Process/server failure
GET /health/ready	Model servis vermeye hazır mı?	Artifact/config load sorunu -> 503
POST /v1/predict	Tek trip duration tahmini	422 contract, 503 unavailable, 500 unexpected
GET /metrics	Prometheus exposition	Platform scrape alarmı
GET /docs	OpenAPI UI	Production policy ile kapatılabilir

11.4 Observability

Middleware her request'e caller-supplied veya UUID request ID bağlar. JSON logs path, latency ve prediction context içerir. Prometheus Counter status bazlı throughput/error, Histogram latency ve prediction distribution sağlar. Ham request persistence yoktur; gerçek sistemde prediction-label join için privacy-reviewed event ID gereklidir.

12. Container ve local observability stack

Multi-stage Docker build önce wheel üretir, sonra minimal runtime image'a yalnızca package, configs ve gate'i geçmiş artifacts kopyalar. Runtime non-root `appuser` kullanır. Compose API root filesystem'ini read-only, `/tmp`'yi tmpfs ve artifact mount'u read-only yapar; privilege escalation kapalıdır.

Servis	Port	Rol
api	8000	Prediction, health ve Prometheus exposition
prometheus	9090	15 saniyede API metric scrape
grafana	3000	Provisioned request-rate ve p95 latency dashboard

Docker executable bu çalışma ortamında bulunmadığı için image runtime build'i burada çalıştırılmaz. Dockerfile, Compose ve config syntax statik olarak doğrulandı; CI workflow gerçek Docker build ve Trivy scan'i zorunlu kılar.

13. CI: code quality ve supply-chain kontrolleri

Pull request ve main push iki job çalıştırır. Quality job format, lint, strict type check, tests/coverage, Bandit ve dependency audit uygular. Runtime, dev ve docs lock'ları ayrı, transitively pinned ve `--require-hashes` ile kurulur. Container job ancak quality sonrası sentetik smoke artifact üretir, digest-pinned Python tabanından image build eder ve high/critical vulnerability için Trivy çalıştırır. Bütün third-party Actions full commit SHA'ya pinlidir. Bu çalışma rehberi ayrıca `make pdf` ile üretilir; PDF build veya byte comparison application CI gate'inin parçası değildir.

Kontrol	Yakaladığı sınıf	Neyi kanıtlamaz
Ruff	style, import, common bug patterns	Business correctness
mypy strict	type contract violations	Runtime data validity
pytest + coverage	asserted behavior regressions	Test edilmemiş behavior
Bandit	yaygın Python security smells	Tam threat model
pip-audit	bilinen dependency CVE'leri	Zero-day veya app logic
Docker build	image'ın üretilebilirliği	Production load capacity
Trivy	image package vulnerabilities	Runtime authorization

14. CD: gated retraining, staging, promotion, rollback

Workflow manual veya monthly schedule ile başlar. Manual run yıl/ay alır; schedule conservative lag adayını gerçek source availability ile doğrular ve gerekirse geriye yürür. Üç aylık window indirilir, her ay doğrulanır, birleşik temporal split train/gate edilir. Artifacts evidence olarak saklanır ve yalnızca başarıdan sonra model dahil image oluşturulur. Image GHCR'a provenance ve SBOM ile push edilir.

- Staging protected environment exact image digest'i deploy eder.
- Rollout tamamlanana kadar beklenir; readiness başarısızsa ilerlemez.
- Smoke test önce readiness sonra gerçek prediction contract'ını çağırır.
- Production environment approval/policy staging başarılı olmadan açılmaz.
- Production aynı digest'i kullanır; rebuild yapılmaz.
- Champion pull/extraction hatası release'i durdurur; karşılaştırmayı atlamak explicit `skip\_champion` ister.
- Kustomize 5.6.0 commit-pinned setup action ile kurulur; runner state'ine güvenilmez.

- Rollback yalnız production apply başarılı olduktan sonraki bir step hata verirse çalışır.

Repo cluster credential saklamaz. OIDC veya platformun scoped deployment integration'ı ayrıca bağlanmalıdır. Environment name kullanmak tek başına approval sağlamaz; GitHub repository settings içinde reviewers ve protection rules tanımlanmalıdır.

15. Kubernetes deployment modeli

Base Deployment iki replica, non-root pod security context, CPU/memory request-limit, readiness ve liveness probe, no privilege escalation, read-only root filesystem ve dropped capabilities içerir. Service port 80'i container 8000'e yönlendirir. Kustomize overlay staging ve production namespace/tag farkını taşır; production replica sayısını üç çıkarır.

15.1 Eksik platform bileşenleri

- Ingress, TLS certificate ve authentication/authorization
- NetworkPolicy ve egress allowlist
- HorizontalPodAutoscaler ve disruption budget
- Secret manager ve workload identity
- Central log/metric retention, alert routing ve SLO dashboards
- Canary/blue-green controller ve automated analysis
- Model registry approval UI ve artifact signature verification

Bunların repo dışı olduğu gizlenmemiştir. Gerçek platform seçilmeden vendor-specific YAML uydurmak yerine açık integration contract bırakılmıştır.

16. Monitoring: operasyon, drift ve delayed labels

16.1 Üç monitoring katmanı

Katman	Ne zaman?	Sinyaller	Aksiyon
Operational	Anında	availability, 5xx, p95 latency, saturation	traffic/infra incident
Data/prediction	Labelsız erken	numeric/categorical PSI, unseen route, prediction shift	investigate, block auto-promotion
Performance	Drop-off label sonrası	MAE, p95 AE, bias, slice metrics	retrain/challenger/rollback

16.2 PSI

Reference profile training split'ten numeric histogramlar, pickup/drop-off/route category dağılımları, known-route seti ve bounded prediction histogramı üretir. Current batch aynı edge/category bucket'larıyla karşılaştırılır. PSI 0.10/0.25 bandları ve unseen-route limiti config'dedir; bunlar evrensel gerçek değil, kalibrasyon başlangıcıdır.

16.3 Neden prediction drift yetmez?

Prediction dağılımı input mix nedeniyle değişebilir ve model hala doğru olabilir. Tersine dağılım sabit görünürken conditional error artabilir. Bu nedenle gecikmeli labels geldiğinde overall ve hour/weekday/route-frequency/zone slice bazında performans hesaplanmalıdır.

17. Test stratejisi

Test	Seviye	Koruduğu davranış
test_config	Unit	Inheritance ve validated settings
test_validation	Unit	Duration, schema ve exact duplicate policy
test_features	Unit	Finite outputs, column order, leakage exclusion, cyclical encoding
test_gates	Unit	Good candidate pass ve açıklayıcı multi-failure
test_ingest/period	Unit	Availability fallback ve rolling window
test_monitoring/drift	Unit	PSI, invalid rate ve category behavior
test_api_schema	Contract	Timezone kabulü ve unknown field rejection
test_training_pipeline	Integration	Single/rolling Parquet'ten artifact, lineage ve metrics
test_api	Integration	Fitted model startup, prediction ve readiness

Doğrulama komutları: `make lint`, `make type`, `make test`, `make security`. Güncel test sayısı ve coverage bu komutların çıktısından okunur; rehber bu değerleri sabit yazmaz.

18. Local çalışma laboratuvarı

18.1 Kurulum

```
python -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
pip install --require-hashes -r requirements-dev.lock
pip install --no-deps -e .
```

18.2 Sentetik smoke data

```
python scripts/generate_sample_data.py --rows 5000 --start 2025-01-01 --output ↵
data/raw/yellow_tripdata_2025-01.parquet
python scripts/generate_sample_data.py --rows 5000 --start 2025-02-01 --output ↵
data/raw/yellow_tripdata_2025-02.parquet
python scripts/generate_sample_data.py --rows 5000 --start 2025-03-01 --output ↵
data/raw/yellow_tripdata_2025-03.parquet
taxi-ml train data/raw/yellow_tripdata_2025-0{1,2,3}.parquet --config-path configs/development.yaml --model- ↵
version local-v1.2
```

Sentetik generator gerçek performans değerlendirmesi değildir. Ama schema, label construction, split, transformer, model, gates ve artifact I/O'nun tamamını hızlı ve deterministik çalıştırır.

18.3 API

```
uvicorn taxi_duration.serving.app:app --reload --port 8000

curl -s http://localhost:8000/health/ready
curl -s -X POST http://localhost:8000/v1/predict \
-H 'Content-Type: application/json' \
-d '{"pickup_datetime":"2025-01-15T08:30:00- ↵
05:00","pickup_location_id":132,"dropoff_location_id":230,"passenger_count":1}'
```

18.4 Resmi data

```
taxi-ml download 2025 1 && taxi-ml download 2025 2 && taxi-ml download 2025 3
taxi-ml train data/raw/yellow_tripdata_2025-01.parquet \
data/raw/yellow_tripdata_2025-02.parquet \
data/raw/yellow_tripdata_2025-03.parquet \
--config-path configs/development.yaml --model-version 2025-q1-v1.2
```

18.5 Drift

```
taxi-ml drift data/raw/current_month.parquet \
--model-path artifacts/model.joblib \
--metadata-path artifacts/metadata.json \
--reference-path artifacts/reference_profile.json \
--output-path reports/monitoring/drift.json --fail-on-critical
```

19. Failure injection çalışmaları

Deney	Nasıl	Beklenen
Leakage guard	API JSON'a fare_amount ekle	422: extra field forbidden
Timezone guard	Offset'siz datetime gönder	422: timezone required
Readiness	Artifact path'i yanlış yap	live 200, ready 503
Data schema	PULocationID kolonunu kaldır	Training başlamadan ValueError
Gate failure	max_test_mae_minutes değerini çok düşür	metrics yazılır, artifact release edilmez
Drift	Yeni PU-DO route'ları ekle	route PSI/unseen rate artar
Staging smoke	Endpoint path'ini boz	Production job başlamaz
Production rollout	Readiness fail image deploy et	rollout fail ve undo

20. Security ve threat model

Tehdit	Mevcut kontrol	Kalan iş
Malicious input	Pydantic strict schema/ranges	rate limit, auth, payload limits
Artifact tampering	trusted CI path + hashes	signature verification, registry policy
Dependency CVE	pip-audit + Trivy	patch SLA, internal mirror
Container escape	non-root, read-only, drop caps	seccomp/AppArmor, node policy
Secret leak	repo secret yok	secret manager + rotation
Supply-chain rebuild	same digest promotion, provenance/SBOM	attestation enforcement
Data privacy	raw request persistence yok	retention, minimization, DPIA
Abusive automation	protected production environment	segregation of duties

21. Production'a geçmeden önce checklist

- ☐ TLC schema ve sample month üzerinde gerçek training tamamlandı.
- ☐ Rejection/out-of-period/duplicate/volume gate threshold'ları gerçek TLC ile kalibre edildi.
- ☐ Hash-locked dependencies ve base image digest sabitlendi.
- ☐ Cluster authentication OIDC ile bağlandı; least privilege test edildi.
- ☐ Production environment reviewers/protection rules tanımlandı.
- ☐ Ingress, TLS, API auth, rate limiting ve network policies çalışıyor.
- ☐ Load test sonucu replica/resource/HPA değerleri ayarlandı.
- ☐ SLO, alert thresholds, on-call route ve runbook ownership belirlendi.
- ☐ Prediction-label event join privacy review'dan geçti.
- ☐ Model artifact signature/attestation doğrulaması enforce edildi.
- ☐ Canary veya blue-green promotion stratejisi test edildi.
- ☐ Rollback tatbikatı ve post-incident evidence retention doğrulandı.

22. v1.2 sonrası teknik yol haritası

Öncelik	Geliştirme	Hipotez
P0	Gerçek TLC batch + schema regression suite	Source drift erken yakalanır
P0	Gerçek TLC benchmark + threshold table	Sentetik kanıt production iddiasına dönüşmez
P1	Taxi-zone geometry/centroid route distance	Unseen route genellemesi artar
P1	Traffic/weather/events challenger	Peak regime error düşer
P1	Quantile models	ETA uncertainty kullanıcıya taşınır
P1	Rare-route and zone slice gates	Aggregate metric körlüğü azalır
P2	Canary analysis + automatic rollback policy	Blast radius düşer
P2	Feature/model registry	Governance ve concurrent experiments gelişir

23. Kaynaklar

- NYC TLC Trip Record Data: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- NYC TLC Yellow Trip Data Dictionary: https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf
- scikit-learn HistGradientBoostingRegressor: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>
- GitHub deployment environments: <https://docs.github.com/en/actions/how-to-deploy/configure-and-manage-deployments/manage-environments>
- Kubernetes probes and pod lifecycle: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- Repository source code and ADRs included with this guide.

Ek A. Tam kaynak kod referansı

Aşağıdaki listing'ler teslim edilen repository'den build sırasında doğrudan okunmuştur. Bu sayede açıklama ile kod arasında kopyalama kaynaklı sürüm farkı oluşmaz. Binary artifacts, cache dosyaları ve üretilmiş coverage çıktıları eklenmemiştir.

A.1 .env.example

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
TAXI_ENVIRONMENT=development
TAXI_CONFIG_PATH=configs/development.yaml
TAXI_MODEL_PATH=artifacts/model.joblib
TAXI_METADATA_PATH=artifacts/metadata.json
TAXI_LOG_LEVEL=INFO
TAXI_MLFLOW_TRACKING_URI=http://mlflow:5000
TAXI_ENABLE_MLFLOW=false
```

A.2 .github/workflows/ci.yml

Her code change için software/supply-chain gate zinciri.

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5
        with:
          python-version: "3.11", cache: pip
      - run: python -m pip install --upgrade pip
      - run: pip install --require-hashes -r requirements-dev.lock
      - run: pip install --no-deps -e .
      - run: ruff format --check .
      - run: ruff check .
      - run: mypy src
      - run: pytest
      - run: bandit -c pyproject.toml -r src
      - run: pip-audit -r requirements-dev.lock
  container:
    runs-on: ubuntu-latest
    needs: quality
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5
        with:
          python-version: "3.11", cache: pip
      - run: pip install --require-hashes -r requirements.lock
      - run: pip install --no-deps -e .
      - run: python scripts/generate_sample_data.py --rows 1000
      - run: taxi-ml train data/raw/sample.parquet --model-version ci-smoke
      - uses: docker/setup-buildx-action@8d2750c68a42422c14e847fe6c8ac0403b4cbd6f # v3
      - uses: docker/build-push-action@10e90e3645eae34f1e60eeb005ba3a3d33f178e8 # v6
        with:
          context: .
          push: false
          load: true
          tags: nyc-taxi-duration:ci
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - uses: aquasecurity/trivy-action@915b19bbe73b92a6cf82a1bc12b087c9a19a5fe2 # v0.28.0
        with:
          image-ref: nyc-taxi-duration:ci
          severity: CRITICAL,HIGH
          exit-code: "1"
      - name: Container smoke test
        run: |
          docker run -d --name api -p 8000:8000 -e TAXI_CONFIG_PATH=/app/configs/development.yaml nyc-taxi-duration:ci
          for i in $(seq 1 30); do curl -sf localhost:8000/health/ready && break; sleep 1; done
          ./scripts/smoke_test.sh http://localhost:8000
```

A.3 .github/workflows/retrain.yml

Data ingestion'dan exact-digest production promotion'a gated ML CD.

```
name: Gated retraining and deployment
on:
```

```

workflow_dispatch:
  inputs:
    year: {description: "TLC data year (blank = automatic lagged month)", required: false, type: string}
    month: {description: "TLC data month, 1-12 (blank = automatic)", required: false, type: string}
    skip_champion:
      description: "Skip champion comparison (first release or deliberate runtime upgrade)"
      required: false
      type: boolean
      default: false
  schedule:
    - cron: "17 6 15 * *"
permissions:
  contents: read
  packages: write
  id-token: write
concurrency:
  group: model-production-deploy
  cancel-in-progress: false
jobs:
  candidate:
    runs-on: ubuntu-latest
    outputs:
      digest: ${ steps.build.outputs.digest }
      image: ${ steps.image.outputs.name }
    steps:
      - uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
      - uses: actions/setup-python@a26af69be951a213d495a4c3e4e4022e16d87065 # v5
      with:
        python-version: "3.11", cache: pip
      - run: pip install --require-hashes -r requirements.lock
      - run: pip install --no-deps -e .
      - name: Image name (GHCRC requires lowercase)
        id: image
        run: echo "name=ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/nyc-taxi-duration" >> "$GITHUB_OUTPUT"
      - name: Resolve data period
        id: period
        # Zero-padding happens in one place (Python), so manual "1" and "01" resolve identically.
        run: taxi-ml resolve-period --year "${ inputs.year }" --month "${ inputs.month }" >> "$GITHUB_OUTPUT"
      - name: Download rolling training window
        run: |
          IFS=' ' read -ra PERIODS <<< "${ steps.period.outputs.window }"
          for period in "${PERIODS[@]}"; do
            taxi-ml download "${period%*-}" "${period#*-}" --config-path configs/staging.yaml
          done
      - uses: docker/login-action@c94ce9fb468520275223c153574b00df6fe4bcc9 # v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - name: Fetch production champion
        id: champion
        if: ${ !inputs.skip_champion }
        run: |
          image="${ steps.image.outputs.name }:production"
          docker pull "$image"
          cid=$(docker create "$image")
          trap 'docker rm -f "$cid" >/dev/null 2>&1 || true' EXIT
          mkdir -p champion
          docker cp "$cid:/app/artifacts/model.joblib" champion/model.joblib
          docker cp "$cid:/app/artifacts/metadata.json" champion/metadata.json
          test -s champion/model.joblib
          test -s champion/metadata.json
          echo "args=--champion-model-path champion/model.joblib --champion-metadata-path champion/metadata.json" >> "$GITHUB_OUTPUT"
      - name: Train and enforce release gates
        env:
          GIT_SHA: ${ github.sha }
        run: |
          IFS=' ' read -ra PERIODS <<< "${ steps.period.outputs.window }"
          DATA_PATHS=()
          for period in "${PERIODS[@]}"; do
            DATA_PATHS+=("data/raw/yellow_tripdata_${period}.parquet")
          done
          taxi-ml train "${DATA_PATHS[@]}" \
            --config-path configs/staging.yaml \
            --model-version ${ steps.period.outputs.period }-${ github.run_id } \
            ${ steps.champion.outputs.args }
      - uses: actions/upload-artifact@ea165f8d65b6e75b540449e92b4886f43607fa02 # v4
        if: always()
        with:
          name: model-candidate-${ github.run_id }
          path: artifacts/
          retention-days: 30
      - uses: docker/setup-buildx-action@8d2750c68a42422c14e847fe6c8ac0403b4cbd6f # v3
      - uses: docker/build-push-action@10e90e3645eae34f1e60eeb005ba3a3d33f178e8 # v6
        id: build
        with:
          context: .
          push: true
          tags: ${ steps.image.outputs.name }:candidate-${ github.run_id }
          provenance: true
          sbom: true
    staging:
      runs-on: ubuntu-latest

```

```

needs: candidate
environment: staging
steps:
- uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
- uses: imranismail/setup-kustomize@2ba527d4d055ab63514ba50a99456fc35684947f # v2.1.0
  with: {kustomize-version: "5.6.0"}
# Cluster credentials (OIDC / scoped deploy integration) must be configured here.
- name: Deploy gated digest to staging
  working-directory: k8s/overlays/staging
  run: |
    kustomize edit set image taxi-duration-api=${{ needs.candidate.outputs.image }}@${{
      needs.candidate.outputs.digest }}
    kubectl apply -k .
    kubectl -n taxi-staging rollout status deployment/taxi-duration-api --timeout=180s
- run: ./scripts/smoke_test.sh "${{ vars.STAGING_BASE_URL }}"

production:
runs-on: ubuntu-latest
needs: [candidate, staging]
environment: production
steps:
- uses: actions/checkout@11d5960a326750d5838078e36cf38b85af677262 # v4
- uses: imranismail/setup-kustomize@2ba527d4d055ab63514ba50a99456fc35684947f # v2.1.0
  with: {kustomize-version: "5.6.0"}
- uses: docker/login-action@c94ce9fb468520275223c153574b00df6fe4bcc9 # v3
  with:
    registry: ghcr.io
    username: ${github.actor}
    password: ${secrets.GITHUB_TOKEN}
- uses: docker/setup-buildx-action@8d2750c68a42422c14e847fe6c8ac0403b4cbd6f # v3
- name: Apply the exact staging digest
  id: production_apply
  working-directory: k8s/overlays/production
  run: |
    kustomize edit set image taxi-duration-api=${{ needs.candidate.outputs.image }}@${{
      needs.candidate.outputs.digest }}
    kubectl apply -k .
- name: Wait for production rollout
  run: |
    kubectl -n taxi-production rollout status deployment/taxi-duration-api --timeout=300s
- run: ./scripts/smoke_test.sh "${{ vars.PRODUCTION_BASE_URL }}"
- name: Record new champion
  # The moving `production` tag is what the next retraining run compares against.
  run: |
    docker buildx imagetools create \
      --tag ${needs.candidate.outputs.image}:production \
      ${needs.candidate.outputs.image}@${needs.candidate.outputs.digest}
- name: Roll back on failure
  if: ${failure()} && steps.production_apply.outcome == 'success'
  run: kubectl -n taxi-production rollout undo deployment/taxi-duration-api

```

A.4 DATA_CARD.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

# Data card

## Source

NYC Taxi & Limousine Commission monthly yellow-taxi trip records in Parquet format. The application downloads
files from the source URL configured in `configs/base.yaml`; it does not redistribute them.

## Unit of observation

One completed yellow-taxi trip record. The training label is elapsed minutes from pickup to drop-off.

## Fields retained by the learning system

- `tpep_pickup_datetime`
- `tpep_dropoff_datetime` (label construction only)
- `PULocationID`
- `DOLocationID`
- `passenger_count`

All other source fields are discarded before model training.

## Known issues

Administrative trip records can contain invalid durations, missing passenger counts, invalid location codes,
exact duplicate rows, and behavior changes across vendors or collection versions. Zone IDs are coarse and do not
encode
the driven route. Timestamps represent New York local operational time in the source.

## Quality controls

Required-column assertion, parse checks, source-month filter (stray rows from other months are removed and
counted as `out_of_period_rows`), configured value ranges including exclusion of zones 264 (Unknown) and
265 (Outside of NYC), exact-row duplicate removal over the five retained source fields, chronological
sorting, release gates for invalid/out-of-period/duplicate rates and valid volume, per-file SHA-256, and
time-range recording. The exact-row rule is deliberately conservative: without a stable source trip ID,
near-duplicates are retained rather than risking removal of legitimate trips with similar attributes.

## Leakage policy

```

Drop-off time is label-only. Realized distance, fare components, tip, total, rate code, payment type, and any post-trip field are prohibited from features. The central feature transformer exposes an auditable allowlist.

Governance

Raw files remain immutable and are excluded from Git. Production retention, access controls, encryption, and privacy review belong to the deploying organization. Do not use this model for punitive individual decisions.

A.5 Dockerfile

Gate'i geçmiş artifact içeren non-root, multi-stage serving image.

```
ARG PYTHON_IMAGE=python:3.11-slim@sha256:9534e5a8e315485d4061ed659af0fd78a284c015f9b73661b41d6bab25604534
FROM ${PYTHON_IMAGE} AS builder
WORKDIR /build
ENV PIP_DISABLE_PIP_VERSION_CHECK=1 PIP_NO_CACHE_DIR=1
# The same lock used for training: the image must unpickle the model with the exact
# scikit-learn/numpy that produced it (also enforced at load time via metadata).
COPY pyproject.toml README.md requirements.lock ./
COPY src ./src
RUN python -m pip wheel --wheel-dir /wheels --require-hashes -r requirements.lock && \
    python -m pip wheel --wheel-dir /wheels --no-deps .

FROM ${PYTHON_IMAGE} AS runtime
ARG APP_UID=10001
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \
    TAXI_CONFIG_PATH=/app/configs/production.yaml \
    TAXI_MODEL_PATH=/app/artifacts/model.joblib \
    TAXI_METADATA_PATH=/app/artifacts/metadata.json
RUN adduser --disabled-password --gecos "" --uid ${APP_UID} appuser
COPY --from=builder /wheels /wheels
RUN python -m pip install --no-cache-dir /wheels/* && rm -rf /wheels
COPY configs ./configs
COPY artifacts/model.joblib artifacts/model.joblib
COPY artifacts/metadata.json artifacts/metadata.json
COPY artifacts/reference_profile.json artifacts/reference_profile.json
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health/live', timeout=2)"
# One worker per container. prometheus_client keeps metrics per process, so several workers
# behind one /metrics endpoint return a different worker's counters on each scrape. Horizontal
# scale comes from Kubernetes replicas, each scraped as its own target.
CMD ["uvicorn", "taxi_duration.serving.app:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1", "--no- ↵
    access-log"]
```

A.6 Makefile

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

.PHONY: install lock pdf format lint type test security sample train evaluate drift serve docker-up docker-down clean

```
install:
    [python -m pip install --require-hashes -r requirements-dev.lock
    [python -m pip install --no-deps -e .

format:
    [ruff format .
    [ruff check --fix .

lint:
    [ruff format --check .
    [ruff check .

type:
    [mypy src

test:
    [pytest

security:
    [bandit -c pyproject.toml -r src
    [pip-audit -r requirements-dev.lock

lock:
    [uv pip compile pyproject.toml --python-version 3.11 --python-platform \
    [ x86_64-manylinux_2_28 --generate-hashes -o requirements.lock --no-header
    [uv pip compile pyproject.toml --extra dev --python-version 3.11 --python-platform \
    [ x86_64-manylinux_2_28 --generate-hashes -o requirements-dev.lock --no-header
    [( echo 'reportlab>=4.2,<5' ) | uv pip compile - --python-version 3.11 --python-platform \
    [ x86_64-manylinux_2_28 --generate-hashes -o requirements-docs.lock --no-header

pdf:
    [python -m pip install --require-hashes -r requirements-docs.lock
    [python tools/generate_pdf.py

sample:
    [python scripts/generate_sample_data.py
    [python scripts/generate_sample_data.py --start 2025-02-01 --seed 7 --output data/raw/sample_next.parquet
```

```

train:
[]taxi-ml train data/raw/sample.parquet --model-version local-sample

evaluate:
[]taxi-ml evaluate data/raw/sample_next.parquet

drift:
[]taxi-ml drift data/raw/sample_next.parquet

serve:
[]uvicorn taxi_duration.serving.app:app --reload --port 8000

docker-up:
[]docker compose up --build

docker-down:
[]docker compose down

clean:
[]find . -type d -name __pycache__ -prune -exec rm -rf {} +
[]rm -rf .pytest_cache .mypy_cache .ruff_cache .coverage htmlcov dist build

```

A.7 configs/base.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

project_name: nyc-taxi-duration
random_seed: 42
timezone: America/New_York
data:
  raw_dir: data/raw
  processed_dir: data/processed
  source_url_template: https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_{year}-{month:02d}.parquet
  min_duration_minutes: 1.0
  max_duration_minutes: 120.0
  min_passenger_count: 1
  max_passenger_count: 6
  # 264 = Unknown, 265 = Outside of NYC in the TLC zone lookup; neither is a routable zone.
  valid_zone_min: 1
  valid_zone_max: 263
  training_window_months: 3
  source_lookback_months: 6
  drop_exact_duplicates: true
split:
  train_fraction: 0.70
  validation_fraction: 0.15
model:
  name: hist_gradient_boosting
  zone_encoding: target
  max_iter: 200
  learning_rate: 0.08
  max_leaf_nodes: 31
  l2_regularization: 1.0
  min_samples_leaf: 30
  target_encoder_cv: 5
gates:
  max_validation_mae_minutes: 9.0
  max_test_mae_minutes: 9.5
  max_p95_absolute_error_minutes: 30.0
  min_improvement_vs_baseline: 0.05
  max_validation_test_gap: 2.0
  # Non-inferiority margin, relative to champion MAE. Must be > 0: with 0 the gate becomes a
  # superiority test and blocks ~95% of retrains that are exactly as good as the champion.
  max_regression_vs_champion: 0.01
  max_rejection_rate: 0.10
  max_out_of_period_rate: 0.01
  max_duplicate_rate: 0.01
  min_valid_rows: 10000
  champion_confidence_level: 0.95
serving:
  host: 0.0.0.0
  port: 8000
  prediction_lower_bound: 1.0
  prediction_upper_bound: 120.0
monitoring:
  reference_path: artifacts/reference_profile.json
  psi_warning: 0.10
  psi_critical: 0.25
  max_invalid_rate: 0.05
  max_unseen_route_rate: 0.20
  performance_window: 1000

```

A.8 configs/development.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

extends: base.yaml
environment: development
model:
  max_iter: 80
  min_samples_leaf: 10

```

```

gates:
  max_validation_mae_minutes: 15.0
  max_test_mae_minutes: 15.0
  min_valid_rows: 100

```

A.9 configs/production.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

extends: base.yaml
environment: production

```

A.10 configs/staging.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

extends: base.yaml
environment: staging

```

A.11 docker-compose.yml

Local API, Prometheus ve Grafana observability stack.

```

services:
  api:
    build: .
    image: nyc-taxi-duration:local
    ports: ["8000:8000"]
    environment:
      TAXI_CONFIG_PATH: /app/configs/development.yaml
      TAXI_MODEL_PATH: /app/artifacts/model.joblib
      TAXI_METADATA_PATH: /app/artifacts/metadata.json
    volumes:
      - ./artifacts:/app/artifacts:ro
    restart: unless-stopped
    read_only: true
    tmpfs: [/tmp]
    security_opt: ["no-new-privileges:true"]
  prometheus:
    image: prom/prometheus:v2.55.1
    ports: ["9090:9090"]
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    depends_on: [api]
  grafana:
    image: grafana/grafana:11.3.1
    ports: ["3000:3000"]
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-change-me}
    volumes:
      - ./monitoring/grafana/provisioning:/etc/grafana/provisioning:ro
    depends_on: [prometheus]

```

A.12 docs/adr/0001-problem-and-feature-contract.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

# ADR 0001: Predict duration from trip-start information

Status: accepted

## Context

The TLC record contains fields produced after a trip ends, including drop-off time, metered distance, fare components, payment type, and totals. Using them would improve offline scores while making the online system impossible or dishonest.

## Decision

The API accepts pickup time, pickup zone, intended drop-off zone, and passenger count. Drop-off time is used only to construct the training label. No fare, realized distance, payment, or drop-off-time field enters the model pipeline.

## Consequences

The model is weaker than a retrospective estimator but matches the stated decision point. Offline and online feature transformations share one scikit-learn Pipeline, preventing training-serving skew.

```

A.13 docs/adr/0002-time-split.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

# ADR 0002: Chronological evaluation

Status: accepted

## Decision

```

Sort records by pickup time, train on the earliest 70%, validate on the next 15%, and reserve the latest 15% as a final test segment. Random splitting is prohibited for release evaluation.

Rationale

Production predicts future trips. A chronological split exposes temporal drift and avoids optimistic mixing of near-identical time regimes across train and test.

A.14 docs/adr/0003-simple-model.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

ADR 0003: HistGradientBoosting is the production candidate

Status: accepted (rationale for the target transform revised in v1.1)

Decision

Use a median DummyRegressor as the mandatory baseline and a scikit-learn HistGradientBoostingRegressor with an absolute-error objective as the candidate. Train on ``loglp(duration)`` and invert with ``expml``.

Rationale

The project demonstrates production ML engineering rather than model novelty. The selected estimator handles nonlinear tabular effects, trains quickly, has a small dependency footprint, and packages with the feature transformer as one artifact.

What is being estimated

Quantiles commute with monotone maps, so minimising absolute error in log space yields ``loglp(median Y | x)``; after ``expml`` the model returns the conditional median. The transform therefore does not change the estimand, and tail robustness comes from the L1 loss itself. The transform is kept because duration effects are largely multiplicative (congestion scales every route), and in log space they become additive, which suits an additive ensemble of trees.

Consequences

- Mean bias on right-skewed durations is expected to be negative; it is monitored, not gated.
- The prediction is appropriate for "typical trip" uses (ETA display). Use cases that aggregate predictions (fleet capacity) need a mean estimator: a sum of medians is not the median of the sum.

A.15 docs/adr/0004-zone-encoding.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

ADR 0004: Target-encode zones and routes

Status: accepted (v1.1)

Context

TLC ``LocationID``s are assigned in the alphabetical order of zone names. In v1.0 they entered the model as numbers, which asserts an ordering that has no geographic meaning; trees can still isolate zones, but only by spending many threshold splits, and the pickup-drop-off interaction (the route) is what drives duration. HistGradientBoosting's native categorical support is capped at 255 categories, below the 263 routable zones.

Decision

Build ``route_id = PU * 1000 + DO`` and target-encode route, pickup zone and drop-off zone with ``sklearn.preprocessing.TargetEncoder``:

- cross-fitted during ``fit`` (5-fold, out-of-fold values), so no training row is encoded with its own label;
- full training statistics at inference;
- empirical-Bayes smoothing (``smooth="auto"``) shrinks sparse routes toward the global mean, while the zone encodings carry the marginal zone effects;
- unseen routes receive the training mean; ``metrics.json`` reports error for seen vs unseen routes.

The encoder lives inside ``TransformedTargetRegressor``, so it encodes ``loglp(duration)``, consistent with ADR 0003. ``model.zone_encoding: ordinal`` keeps the v1.0 arm for comparison.

Evidence so far

On the synthetic generator (latent zone geography, 20k rows) test MAE fell from 10.53 to 7.21 minutes. This only shows that the mechanism works when geography matters; the real-data comparison must be run on TLC files and recorded here before the decision is considered validated.

Alternatives considered

Static zone-centroid coordinates and centroid distance (from the TLC zone shapefile) are leakage-safe and cheap, and remain a valid next challenger. Target encoding was chosen first because it learns realised route durations directly instead of a straight-line proxy for them. Precisely, it learns the smoothed mean of ``loglp(duration)`` per route, i.e. a geometric-mean-type centre; that equals the route median only when log-duration is symmetric within the route. The model's final estimand is still the conditional median (L1 loss, ADR 0003); this note concerns only what the encoded feature represents.

A.16 docs/adr/0005-release-comparison-and-runtime.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

ADR 0005: Champion comparison, iteration selection, pinned runtime

Status: accepted (v1.1), comparison rule refined by ADR 0006 in v1.2

Decisions

1. **Champion-challenger gate.** The retraining workflow extracts the model from the image tagged `production` and scores it on the candidate's test window. The candidate may not be worse than the champion by more than `gates.max\_regression\_vs\_champion`. Absolute limits alone would allow a candidate that passes every threshold but is worse than what is serving. A champion that exists but cannot be evaluated blocks the release; no champion (first release) is a recorded skip.
2. **Iteration count on the chronological validation set.** Built-in early stopping holds out a random 10% of the training rows, which mixes periods inside a chronological design. The pipeline fits the maximum number of iterations, reads the validation MAE curve from `staged\_predict`, and refits at the argmin. The test segment is not involved.
3. **One lock file for training, CI, the image and the champion.** A joblib artifact can only be trusted with the artifact-sensitive library versions that wrote it. The hash-locked runtime set is the single resolution; the loader also compares versions recorded in metadata and refuses mismatches. Changing the lock is a deliberate release that runs with `skip\_champion`.

Not adopted

Refitting on train+validation after gating would put the most recent days into the weights, at the cost of shipping an artifact that is not the one evaluated. v1.1 ships the evaluated artifact.

A.17 docs/adr/0006-v12-release-reliability.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

ADR 0006: Availability-aware rolling training and unified release behavior

Status: accepted (v1.2)

Context

A fixed calendar lag can point at an unpublished TLC file. One-month training overfits recent seasonality, quality statistics were reported but did not block release, and raw training predictions differed from the bounded predictions served by the API. The drift report also omitted the categorical route signals that carry most of the model's information.

Decision

- Probe the candidate source object and walk backwards for at most `source\_lookback\_months`.
- Train scheduled candidates on `training\_window\_months` separately validated monthly files, then split the merged rows chronologically.
- Gate rejection, out-of-period, exact-duplicate, and valid-row volume before artifact creation.
- Route every model score through `predict\_duration`, which applies the production bounds.
- Compare candidate and champion on identical rows using the one-sided confidence bound of paired absolute error differences. Permit only the configured relative regression tolerance.
- Monitor numeric, zone, route, unseen-route, and prediction drift.
- Hash resolved configuration and the checksum manifest of every training source file.

Consequences

Retraining fails closed when publication or quality expectations are not met. Seasonal coverage improves at the cost of more download, compute, and storage. (v1.2.1 correction) As first shipped, the confidence gate used a zero margin, which made it a superiority test: a candidate exactly as good as the champion was blocked with probability about 0.95 (0.5 for the v1.1 point-estimate rule). ADR 0007 replaces it with a non-inferiority rule and an hour-clustered standard error. The normal approximation should still be replaced with a block bootstrap if strong temporal autocorrelation is observed. Exact duplicate removal avoids inventing a trip identity; near-duplicates remain.

A.18 docs/adr/0007-champion-noninferiority.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

ADR 0007: Champion comparison is a non-inferiority test with hour-clustered errors

Status: accepted (v1.2.1), corrects ADR 0006

Problem

The question a release gate asks is: "is the candidate at least as good as what is serving?" For paired per-row differences $d_i = |y_i - \text{candidate}_i| - |y_i - \text{champion}_i|$ (positive = candidate worse) with mean d and standard error SE , v1.2 blocked when $d + z \cdot SE > m$, with margin $m = 0$. Passing then requires $d < -z \cdot SE$, i.e. statistically significant improvement*. That is a superiority test. If the two models are equally good ($E[d] = 0$), the pass probability is $P(d < -z \cdot SE) = 1 - \Phi(z) = 0.05$ at 95% confidence, independent of sample size. Measured: 5 of 6 equal-quality retrains blocked in the pipeline; 94.8% ($n = 3,000$) and 96.3% ($n = 300,000$) blocked in simulation. The consequence is a frozen champion whenever the data brings no provable gain, which is the normal case for a stable process.

Decision

1. **Non-inferiority.** Keep the upper bound but require a positive margin: block when $d + z \cdot SE > m$, $m = \text{max_regression_vs_champion} \times \text{champion MAE}$, default 1%, validated $0 < m < 20\%$. The burden of proof stays on the candidate (promotion is the risky action), but "as good as" now passes. As the test set grows, $SE \rightarrow 0$ and the rule converges to "no worse than the margin".
2. **Cluster-robust SE** by pickup hour. Trips in the same hour share traffic, so their d_i are correlated and the i.i.d. SE understates uncertainty. With clusters g of the test rows, $\text{Var}(d) = G/(G-1) \cdot \sum_g (\sum_{i \in g} (d_i - d))^2 / n^2$. Fewer than two clusters is a failed comparison.
3. The report records `mae\_delta`, its SE, the upper bound, the margin in minutes, and the cluster count.

Evidence

Same data, seed-only differences, 20k synthetic rows (51 hourly clusters in the test window): 10 of 12 retrains pass; the two blocked have the largest mean deltas (+0.054, +0.035 min against a 0.072 min margin). Unit simulation with an hour-level shared component: ~83% of equal candidates pass versus ~5% under the zero margin; a candidate 0.5 min worse per trip is blocked.

Trade-offs

The margin is a business tolerance, not a statistical quantity; 1% of MAE (~4 s on a 7-minute MAE) should be revisited with the real-data benchmark. Hour clusters assume independence across hours; day-level or block bootstrap inference remains the next step if cross-hour correlation is material.

A.19 docs/model-card.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

Model card: NYC taxi trip duration

Intended use

Estimate trip duration in minutes at trip start for routing, customer information, or operational planning.

Not intended for

Driver compensation, enforcement, individual performance ranking, emergency decisions, or use outside NYC without retraining and validation.

Inputs and output

Inputs: timezone-aware pickup timestamp, TLC pickup zone ID (1-263), intended TLC drop-off zone ID (1-263), passenger count.

Output: conditional *median* duration in minutes, clipped to the validated 1-120 minute operating range. It is not a mean: predictions should not be summed to plan aggregate capacity (see ADR 0003).

Evaluation

Chronological train/validation/test split across a three-month rolling source window, after validating each file against its own month. The final temporal tail remains validation/test data. Boosting iterations are selected on the validation segment. Release gates cover data quality, MAE, p95 absolute error, improvement over a median baseline, validation-to-test stability, and no regression against the production champion at the configured one-sided paired confidence level on the same test window.

`taxi-ml evaluate` scores a released model on later months;

error is broken down by hour, weekday and whether the route was seen in training. Metrics are written by every training run rather than hard-coded in this card.

Limitations

No live traffic, weather, event, road-closure, or route geometry signal is included. Zone IDs are coarse. TLC records can contain reporting errors. The model can degrade under regime changes. Routes absent from training fall back to zone-level and global statistics and have visibly higher error.

A.20 docs/runbook.md

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

Operations runbook

Symptoms and first checks

1. `/health/live` fails: container or process failure. Inspect platform events and application logs.
2. `/health/ready` returns 503: artifact missing, corrupt, or written by a different Python/scikit-learn/NumPy/pandas/joblib version ('model_load_failed' log line names the mismatch). Do not route traffic; rebuild from the lock.
3. Error-rate alert: correlate request IDs, HTTP status labels, schema errors, and recent deployments.
4. Latency alert: inspect replica saturation, CPU/memory throttling, and prediction histogram volume.
5. Drift 'critical': feature/categorical/prediction PSI exceeds its limit, invalid-value rate exceeds 'max_invalid_rate', or unseen-route rate exceeds 'max_unseen_route_rate'. The report separates the two. A high invalid rate is an upstream data problem, not model drift: fix ingestion first. For PSI, quarantine automatic promotion, inspect changed fields and compare performance once labels arrive.
6. MAE breach after labels arrive: stop promotion, retain champion, train a challenger, and diagnose by time, route, and zone.

Rollback

The production deployment uses an immutable image digest. `kubectrl rollout undo deployment/taxi-duration-api` restores the previous ReplicaSet. Confirm readiness and execute `scripts/smoke\_test.sh` against production. The workflow invokes rollback only if the production manifest was successfully applied; failures before that point leave the healthy deployment untouched. Rollback does not delete the failed image or model; retain both for incident analysis.

Metrics look erratic

Counters that jump back and forth between scrapes mean more than one process is serving one `/metrics` endpoint. The image runs one worker per container by design; check that no override added `--workers`.

Model promotion

Only artifacts produced by the retraining workflow are eligible. Automatic runs first probe TLC publication,

fall back month-by-month within the configured bound, download the complete rolling window, and fail closed if none is available. The workflow compares each candidate against the image tagged `production` and moves that tag only after a healthy production rollout. Run with `skip\_champion` only for the first release or a deliberate runtime-lock upgrade, and record why. Without that explicit input, any production-image pull or artifact extraction failure stops the release; infrastructure failure is never interpreted as absence of a champion. A human reviews data-quality results, time ranges, baseline comparison, validation/test metrics, drift, and metadata. Promotion reuses the exact tested digest. Never rebuild between staging and production. A data-quality gate failure is not waived by good MAE; investigate source publication/schema behavior first.

A.21 k8s/base/deployment.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taxi-duration-api
spec:
  replicas: 2
  selector:
    matchLabels: {app: taxi-duration-api}
  template:
    metadata:
      labels: {app: taxi-duration-api}
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8000"
      prometheus.io/path: /metrics
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: api
          # Logical image name. The CD workflow pins it to an immutable registry digest with
          # `kustomize edit set image taxi-duration-api=<registry>/<image>@sha256:...`.
          image: taxi-duration-api
          ports: [{name: http, containerPort: 8000}]
          env:
            - {name: TAXI_CONFIG_PATH, value: /app/configs/production.yaml}
          resources:
            requests: {cpu: 100m, memory: 256Mi}
            limits: {cpu: 1000m, memory: 1Gi}
          readinessProbe:
            httpGet: {path: /health/ready, port: http}
            initialDelaySeconds: 10
          livenessProbe:
            httpGet: {path: /health/live, port: http}
            initialDelaySeconds: 20
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: ["ALL"]}
          volumeMounts:
            - {name: tmp, mountPath: /tmp}
      volumes:
        - name: tmp
          emptyDir: {sizeLimit: 64Mi}
```

A.22 k8s/base/kustomization.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
resources: [deployment.yaml, service.yaml]
```

A.23 k8s/base/service.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
apiVersion: v1
kind: Service
metadata:
  name: taxi-duration-api
spec:
  selector: {app: taxi-duration-api}
  ports: [{name: http, port: 80, targetPort: http}]
```

A.24 k8s/overlays/production/kustomization.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
resources: [../base]
namespace: taxi-production
# images: set at deploy time by the CD workflow (digest-pinned); do not hand-edit.
patches:
  - patch: |-
    - op: replace
      path: /spec/replicas
      value: 3
  target:
```

```
kind: Deployment
name: taxi-duration-api
```

A.25 k8s/overlays/staging/kustomization.yaml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
resources: [../../base]
namespace: taxi-staging
# images: set at deploy time by the CD workflow (digest-pinned); do not hand-edit.
```

A.26 monitoring/grafana/provisioning/dashboards/provider.yml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
apiVersion: 1
providers:
- name: taxi
  folder: Taxi ML
  type: file
  options:
    path: /etc/grafana/provisioning/dashboards
```

A.27 monitoring/grafana/provisioning/datasources/prometheus.yml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
apiVersion: 1
datasources:
- name: Prometheus
  uid: prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
```

A.28 monitoring/prometheus.yml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
global:
  scrape_interval: 15s
scrape_configs:
- job_name: taxi-api
  metrics_path: /metrics
  static_configs:
  - targets: ["api:8000"]
```

A.29 pyproject.toml

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
[build-system]
requires = ["hatchling>=1.25"]
build-backend = "hatchling.build"

[project]
name = "nyc-taxi-duration-mlops"
version = "1.2.3"
description = "Production-oriented NYC yellow taxi trip-duration regression system"
readme = "README.md"
requires-python = ">=3.11,<3.13"
license = {text = "MIT"}
authors = [{name = "Ugur Alkan"}]
dependencies = [
    "fastapi>=0.115,<1",
    "httpx[socks]>=0.27,<1",
    "joblib>=1.4,<2",
    "numpy>=1.26,<3",
    "pandas>=2.2,<3",
    "prometheus-client>=0.21,<1",
    "pyarrow>=23,<24",
    "pydantic>=2.9,<3",
    "pydantic-settings>=2.5,<3",
    "PyYAML>=6,<7",
    "scikit-learn>=1.5,<1.7",
    "structlog>=24.4,<26",
    "typer>=0.12,<1",
    "uvicorn[standard]>=0.30,<1",
]

[project.optional-dependencies]
docs = ["reportlab>=4.2,<5"]
tracking = ["mlflow>=2.17,<3"]
dev = [
```

```

"bandit[toml]>=1.7,<2",
"mypy>=1.12,<2",
"pandas-stubs>=2.2",
"pip-audit>=2.7,<3",
"pytest>=9.0.3,<10",
"pytest-cov>=5,<7",
"ruff>=0.7,<1",
"types-PyYAML>=6.0",
]

[project.scripts]
taxi-ml = "taxi_duration.cli:app"

[tool.hatch.build.targets.wheel]
packages = ["src/taxi_duration"]

[tool.pytest.ini_options]
addopts = "-q --strict-markers --cov=taxi_duration --cov-report=term-missing --cov-fail-under=75"
testpaths = ["tests"]
markers = ["integration: tests crossing component boundaries"]

[tool.ruff]
line-length = 100
target-version = "py311"
extend-exclude = ["tools/generate_pdf.py"]

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM", "RUF"]

[tool.mypy]
python_version = "3.11"
strict = true
warn_unused_ignores = true
exclude = ["tests"]
disable_error_code = ["misc"]

[[tool.mypy.overrides]]
module = ["joblib", "sklearn", "sklearn.*"]
ignore_missing_imports = true

[tool.bandit]
exclude_dirs = ["tests"]

```

A.30 scripts/generate_sample_data.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""Deterministic synthetic TLC-shaped data for tests, CI and the quick start.

Zones get random latent coordinates. Duration depends on the distance between those
coordinates, not on the difference of zone IDs: real TLC LocationIDs follow the alphabetical
order of zone names, so an ID-difference signal would flatter an ordinal encoding that has no
geographic meaning on real data. Time effects are multiplicative, as they are for traffic.
"""

from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import pandas as pd

N_ZONES = 263

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--rows", type=int, default=20000)
    parser.add_argument("--start", default="2025-01-01")
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--output", type=Path, default=Path("data/raw/sample.parquet"))
    args = parser.parse_args()

    geography = np.random.default_rng(0) # fixed across files: zones do not move
    coords = geography.uniform(0.0, 12.0, size=(N_ZONES + 1, 2)) # km, index = zone id

    rng = np.random.default_rng(args.seed)
    pickup = pd.date_range(args.start, periods=args.rows, freq="1min")
    popularity = geography.dirichlet(np.full(N_ZONES, 0.1)) # few busy zones, long tail
    pu = rng.choice(np.arange(1, N_ZONES + 1), size=args.rows, p=popularity)
    do = rng.choice(np.arange(1, N_ZONES + 1), size=args.rows, p=popularity)
    passenger = rng.integers(1, 7, args.rows)

    distance_km = np.linalg.norm(coords[pu] - coords[do], axis=1) * 1.3 # street detour factor
    hour = pickup.hour.to_numpy()
    rush = ((hour >= 7) & (hour <= 10)) | ((hour >= 16) & (hour <= 20))
    weekend = pickup.dayofweek.to_numpy() >= 5
    speed_kmh = 22.0 * np.where(rush, 0.7, 1.0) * np.where(weekend, 1.15, 1.0)
    duration = (3.0 + 60.0 * distance_km / speed_kmh) * rng.lognormal(0.0, 0.25, args.rows)
    duration = np.clip(duration, 1.1, 115.0)

```

```

frame = pd.DataFrame(
    {
        "tpep_pickup_datetime": pickup,
        "tpep_dropoff_datetime": pickup + pd.to_timedelta(duration, unit="m"),
        "PULocationID": pu,
        "DOLocationID": do,
        "passenger_count": passenger,
    }
)
args.output.parent.mkdir(parents=True, exist_ok=True)
frame.to_parquet(args.output, index=False)
print(args.output)

if __name__ == "__main__":
    main()

```

A.31 scripts/smoke_test.sh

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

#!/usr/bin/env sh
set -eu
base_url="{1:?base URL required}"
curl --fail --silent --show-error "${base_url}/health/ready"
curl --fail --silent --show-error \
  -H 'Content-Type: application/json' \
  -d '{"pickup_datetime":"2025-01-15T08:30:00-05:00","pickup_location_id":132,"dropoff_location_id":230,"passenger_count":1}' \
  "${base_url}/v1/predict"

```

A.32 src/taxi_duration/__init__.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""NYC taxi trip-duration production ML package."""

__version__ = "1.2.3"

```

A.33 src/taxi_duration/cli.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from __future__ import annotations

import json
from datetime import date
from pathlib import Path
from typing import Annotated

import pandas as pd
import typer

from taxi_duration.config import load_config
from taxi_duration.data.ingest import download_month, latest_available_period
from taxi_duration.data.validation import Period
from taxi_duration.logging import configure_logging
from taxi_duration.models.artifact import load_artifact
from taxi_duration.monitoring.drift import drift_report
from taxi_duration.training.pipeline import evaluate_artifact, train_from_parquet

app = typer.Typer(no_args_is_help=True)

def _period(text: str | None) -> Period | None:
    return Period.parse(text) if text else None

def default_period(today: date, lag_months: int = 2) -> Period:
    """Conservative source month: TLC publishes monthly files with a lag of roughly two months."""
    index = today.year * 12 + (today.month - 1) - lag_months
    return Period(index // 12, index % 12 + 1)

def period_window(end: Period, months: int) -> list[Period]:
    if months < 1:
        raise ValueError("months must be at least 1")
    return [end.shift(offset) for offset in range(-(months - 1), 1)]

@app.command("resolve-period")
def resolve_period(
    year: str = "",
    month: str = "",
    lag_months: int = 2,
    config_path: Path = Path("configs/staging.yaml"),
) -> None:
    """Resolve an explicit period or probe backwards to the newest published TLC month."""
    if bool(year.strip()) != bool(month.strip()):
        raise typer.BadParameter("provide both --year and --month, or neither")

```

```

config = load_config(config_path)
period = (
    Period(int(year), int(month))
    if year.strip()
    else latest_available_period(
        default_period(date.today(), lag_months),
        config.data.source_url_template,
        config.data.source_lookback_months,
    )
)
window = period_window(period, config.data.training_window_months)
typer.echo(
    f"year={period.year:04d}\nmonth={period.month:02d}\nperiod={period}\n"
    f"window={','.join(map(str, window))}"
)

@app.command()
def download(
    year: int,
    month: int,
    config_path: Path = Path("configs/development.yaml"),
    overwrite: bool = False,
) -> None:
    """Download one immutable monthly TLC parquet file."""
    config = load_config(config_path)
    path, checksum = download_month(
        year, month, config.data.raw_dir, config.data.source_url_template, overwrite=overwrite
    )
    typer.echo(json.dumps({"path": str(path), "sha256": checksum}))

@app.command()
def train(
    data_paths: Annotated[
        list[Path], typer.Argument(help="One or more chronological monthly Parquet files.")
    ],
    config_path: Path = Path("configs/development.yaml"),
    model_path: Path = Path("artifacts/model.joblib"),
    metadata_path: Path = Path("artifacts/metadata.json"),
    metrics_path: Path = Path("artifacts/metrics.json"),
    reference_path: Path = Path("artifacts/reference_profile.json"),
    model_version: str = "local",
    period: Annotated[
        str | None,
        typer.Option(help="Source month YYYY-MM. Defaults to the month in the file name."),
    ] = None,
    champion_model_path: Annotated[
        Path | None, typer.Option(help="Current production model.joblib.")
    ] = None,
    champion_metadata_path: Annotated[
        Path | None, typer.Option(help="Current production metadata.json.")
    ] = None,
) -> None:
    """Train, evaluate, gate (incl. champion comparison) and package a candidate model."""
    configure_logging()
    config = load_config(config_path)
    metrics = train_from_parquet(
        data_paths,
        config,
        config_path,
        model_path,
        metadata_path,
        metrics_path,
        model_version,
        period=_period(period),
        champion_model_path=champion_model_path,
        champion_metadata_path=champion_metadata_path,
        reference_path=reference_path,
    )
    typer.echo(json.dumps(metrics, indent=2))

@app.command()
def evaluate(
    data_path: Path,
    config_path: Path = Path("configs/development.yaml"),
    model_path: Path = Path("artifacts/model.joblib"),
    metadata_path: Path = Path("artifacts/metadata.json"),
    output_path: Path = Path("reports/evaluation.json"),
    period: str | None = None,
) -> None:
    """Score a trained artifact on a later month (out-of-period generalisation)."""
    configure_logging()
    report = evaluate_artifact(
        data_path, load_config(config_path), model_path, metadata_path, _period(period)
    )
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    typer.echo(json.dumps(report, indent=2))

@app.command()

```

```

def drift(
    data_path: Path,
    config_path: Path = Path("configs/development.yaml"),
    reference_path: Path = Path("artifacts/reference_profile.json"),
    output_path: Path = Path("reports/monitoring/drift.json"),
    fail_on_critical: bool = False,
    model_path: Path = Path("artifacts/model.joblib"),
    metadata_path: Path = Path("artifacts/metadata.json"),
    prediction_drift: bool = True,
) -> None:
    """Compare a raw batch with feature, route, prediction and data-quality references."""
    config = load_config(config_path)
    model = None
    if prediction_drift:
        model, _ = load_artifact(model_path, metadata_path)
    report = drift_report(
        pd.read_parquet(data_path),
        reference_path,
        config.monitoring,
        model=model,
        serving=config.serving if model is not None else None,
    )
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    typer.echo(json.dumps(report, indent=2))
    if fail_on_critical and report["status"] == "critical":
        raise typer.Exit(code=2)

if __name__ == "__main__":
    app()

```

A.34 src/taxi_duration/config.py

Pydantic tabanlı strict configuration schema, inheritance merge ve YAML loader.

```

from __future__ import annotations

import json
from copy import deepcopy
from pathlib import Path
from typing import Any, Literal

import yaml
from pydantic import BaseModel, ConfigDict, Field, model_validator

class StrictModel(BaseModel):
    model_config = ConfigDict(extra="forbid")

class DataConfig(StrictModel):
    raw_dir: Path
    processed_dir: Path
    source_url_template: str
    min_duration_minutes: float = 1.0
    max_duration_minutes: float = 120.0
    min_passenger_count: int = 1
    max_passenger_count: int = 6
    valid_zone_min: int = 1
    valid_zone_max: int = 265
    training_window_months: int = Field(default=3, ge=1, le=12)
    source_lookback_months: int = Field(default=6, ge=1, le=24)
    drop_exact_duplicates: bool = True

class SplitConfig(StrictModel):
    train_fraction: float = Field(gt=0, lt=1)
    validation_fraction: float = Field(gt=0, lt=1)

    @model_validator(mode="after")
    def fractions_leave_test_data(self) -> SplitConfig:
        if self.train_fraction + self.validation_fraction >= 1:
            raise ValueError("train_fraction + validation_fraction must be < 1")
        return self

class ModelConfig(StrictModel):
    name: str
    zone_encoding: Literal["target", "ordinal"] = "target"
    target_encoder_cv: int = Field(default=5, ge=2)
    max_iter: int = Field(gt=0)
    learning_rate: float = Field(gt=0)
    max_leaf_nodes: int = Field(gt=1)
    l2_regularization: float = Field(ge=0)
    min_samples_leaf: int = Field(gt=0)

class GateConfig(StrictModel):
    max_validation_mae_minutes: float
    max_test_mae_minutes: float
    max_p95_absolute_error_minutes: float

```

```

min_improvement_vs_baseline: float = Field(ge=0)
max_validation_test_gap: float
max_regression_vs_champion: float = Field(default=0.01, gt=0, lt=0.2)
max_rejection_rate: float = Field(default=0.10, ge=0, le=1)
max_out_of_period_rate: float = Field(default=0.01, ge=0, le=1)
max_duplicate_rate: float = Field(default=0.01, ge=0, le=1)
min_valid_rows: int = Field(default=10_000, ge=1)
champion_confidence_level: float = Field(default=0.95, gt=0.5, lt=1)

class ServingConfig(StrictModel):
    host: str
    port: int
    prediction_lower_bound: float
    prediction_upper_bound: float

class MonitoringConfig(StrictModel):
    reference_path: Path
    psi_warning: float
    psi_critical: float
    max_invalid_rate: float = Field(default=0.05, ge=0, le=1)
    max_unseen_route_rate: float = Field(default=0.20, ge=0, le=1)
    performance_window: int

class AppConfig(StrictModel):
    project_name: str
    environment: str = "development"
    random_seed: int
    timezone: str
    data: DataConfig
    split: SplitConfig
    model: ModelConfig
    gates: GateConfig
    serving: ServingConfig
    monitoring: MonitoringConfig

def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    result = deepcopy(base)
    for key, value in override.items():
        if isinstance(value, dict) and isinstance(result.get(key), dict):
            result[key] = _deep_merge(result[key], value)
        else:
            result[key] = value
    return result

def load_config(path: str | Path) -> AppConfig:
    config_path = Path(path)
    payload = yaml.safe_load(config_path.read_text(encoding="utf-8")) or {}
    parent = payload.pop("extends", None)
    if parent:
        base_path = config_path.parent / str(parent)
        base_payload = yaml.safe_load(base_path.read_text(encoding="utf-8")) or {}
        payload = _deep_merge(base_payload, payload)
    return AppConfig.model_validate(payload)

def canonical_config_json(config: AppConfig) -> str:
    """Canonical, fully resolved configuration used for artifact lineage hashing."""
    return json.dumps(config.model_dump(mode="json"), sort_keys=True, separators=(",", ":"))

```

A.35 src/taxi_duration/data/__init__.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
"""Data ingestion and validation."""
```

A.36 src/taxi_duration/data/ingest.py

Streaming, atomik TLC month download ve SHA-256 identity.

```

from __future__ import annotations

import hashlib
import time
from pathlib import Path

import httpx
import structlog

from taxi_duration.data.validation import Period

LOGGER = structlog.get_logger(__name__)

NOT_PUBLISHED = frozenset({403, 404}) # CloudFront/S3 answer 403 for a missing object

class SourceProbeError(RuntimeError):

```

```

"""The source could not be probed; availability is unknown, not negative."""

def _probe_status(url: str, timeout_seconds: float) -> int:
    response = httpx.head(url, follow_redirects=True, timeout=timeout_seconds)
    if response.status_code != 405:
        return response.status_code
    with httpx.stream(
        "GET",
        url,
        headers={"Range": "bytes=0-0"},
        follow_redirects=True,
        timeout=timeout_seconds,
    ) as range_response:
        return range_response.status_code

def source_available(
    period: Period,
    url_template: str,
    timeout_seconds: float = 20.0,
    attempts: int = 3,
    backoff_seconds: float = 2.0,
) -> bool:
    """Return whether the immutable TLC object exists; calendar lag alone is not proof.

    Only 200/206 mean published and only 403/404 mean not published. Network errors and any
    other status are retried and then raised, so a transient outage cannot silently move the
    training window to an older month.
    """
    url = url_template.format(year=period.year, month=period.month)
    last_problem = "no attempt made"
    for attempt in range(1, attempts + 1):
        try:
            status = _probe_status(url, timeout_seconds)
        except httpx.HTTPError as exc:
            last_problem = f"{type(exc).__name__}: {exc}"
        else:
            if status in {200, 206}:
                return True
            if status in _NOT_PUBLISHED:
                return False
            last_problem = f"unexpected HTTP status {status}"
            LOGGER.warning("source_probe_retry", url=url, attempt=attempt, problem=last_problem)
            if attempt < attempts:
                time.sleep(backoff_seconds * attempt)
            raise SourceProbeError(f"could not determine availability of {url}: {last_problem}")

def latest_available_period(
    start: Period,
    url_template: str,
    max_lookback_months: int,
) -> Period:
    """Probe backwards from *start* and select the newest actually published source file."""
    for offset in range(max_lookback_months):
        candidate = start.shift(-offset)
        if source_available(candidate, url_template):
            return candidate
    raise RuntimeError(
        f"no published TLC source found from {start} across {max_lookback_months} months"
    )

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

def download_month(
    year: int,
    month: int,
    destination_dir: Path,
    url_template: str,
    *,
    overwrite: bool = False,
) -> tuple[Path, str]:
    if year < 2009 or month not in range(1, 13):
        raise ValueError("year must be >= 2009 and month must be 1..12")
    destination_dir.mkdir(parents=True, exist_ok=True)
    destination = destination_dir / f"yellow_tripdata_{year}-{month:02d}.parquet"
    if destination.exists() and not overwrite:
        return destination, sha256_file(destination)

    url = url_template.format(year=year, month=month)
    temporary = destination.with_suffix(".parquet.part")
    LOGGER.info("download_started", url=url, destination=str(destination))
    try:
        with httpx.stream("GET", url, follow_redirects=True, timeout=120.0) as response:
            response.raise_for_status()

```

```

        with temporary.open("wb") as output:
            for chunk in response.iter_bytes(chunk_size=1024 * 1024):
                output.write(chunk)
            temporary.replace(destination)
    except Exception:
        temporary.unlink(missing_ok=True)
        raise
    checksum = sha256_file(destination)
    LOGGER.info("download_completed", bytes=destination.stat().st_size, sha256=checksum)
    return destination, checksum

```

A.37 src/taxi_duration/data/validation.py

Source schema allowlist, label construction, domain filtering ve quality report.

```

from __future__ import annotations

import re
from dataclasses import dataclass
from pathlib import Path

import pandas as pd

from taxi_duration.config import DataConfig

PICKUP = "tpep_pickup_datetime"
DROPOFF = "tpep_dropoff_datetime"
PULOCATION = "PULocationID"
DOLOCATION = "DOLocationID"
PASSENGERS = "passenger_count"
REQUIRED_COLUMNS = (PICKUP, DROPOFF, PULOCATION, DOLOCATION, PASSENGERS)

_PERIOD_IN_NAME = re.compile(r"(\d{4})-(\d{2})\.parquet$")

@dataclass(frozen=True)
class Period:
    """One calendar month of source data, in naive New York local time like the TLC files."""

    year: int
    month: int

    def __post_init__(self) -> None:
        if self.year < 2009 or not 1 <= self.month <= 12:
            raise ValueError("period must be year >= 2009 and month 1..12")

    @property
    def start(self) -> pd.Timestamp:
        return pd.Timestamp(year=self.year, month=self.month, day=1)

    @property
    def end(self) -> pd.Timestamp:
        return self.start + pd.offsets.MonthBegin(1)

    def __str__(self) -> str:
        return f"{self.year:04d}-{self.month:02d}"

    def shift(self, months: int) -> Period:
        index = self.year * 12 + self.month - 1 + months
        return Period(index // 12, index % 12 + 1)

    @classmethod
    def parse(cls, text: str) -> Period:
        match = re.fullmatch(r"(\d{4})-(\d{1,2})", text.strip())
        if not match:
            raise ValueError(f"period must look like YYYY-MM, got {text!r}")
        return cls(int(match.group(1)), int(match.group(2)))

    @classmethod
    def from_filename(cls, path: Path) -> Period | None:
        match = _PERIOD_IN_NAME.search(path.name)
        return cls(int(match.group(1)), int(match.group(2))) if match else None

@dataclass(frozen=True)
class ValidationReport:
    input_rows: int
    valid_rows: int
    rejected_rows: int
    rejection_rate: float
    out_of_period_rows: int
    duplicate_rows: int
    period: str | None

    @property
    def out_of_period_rate(self) -> float:
        return self.out_of_period_rows / self.input_rows if self.input_rows else 0.0

    @property
    def duplicate_rate(self) -> float:
        return self.duplicate_rows / self.input_rows if self.input_rows else 0.0

```

```

def validate_and_clean(
    raw: pd.DataFrame, config: DataConfig, period: Period | None = None
) -> tuple[pd.DataFrame, ValidationReport]:
    """Apply schema, domain and (optionally) source-period rules.

    Monthly TLC files contain a small number of rows whose pickup timestamp falls outside the
    file's month. Without a period filter those rows sort to the extremes of the chronological
    split and corrupt both the test window and the recorded lineage periods.
    """
    missing = sorted(set(REQUIRED_COLUMNS) - set(raw.columns))
    if missing:
        raise ValueError(f"missing required columns: {missing}")

    frame = raw[list(REQUIRED_COLUMNS)].copy()
    frame[PICKUP] = pd.to_datetime(frame[PICKUP], errors="coerce")
    frame[DROPOFF] = pd.to_datetime(frame[DROPOFF], errors="coerce")
    frame["duration_minutes"] = (frame[DROPOFF] - frame[PICKUP]).dt.total_seconds() / 60

    in_period = pd.Series(True, index=frame.index)
    if period is not None:
        in_period = (frame[PICKUP] >= period.start) & (frame[PICKUP] < period.end)

    duplicates = frame.duplicated(subset=list(REQUIRED_COLUMNS), keep="first")
    valid = (
        frame[PICKUP].notna()
        & frame[DROPOFF].notna()
        & in_period
        & frame["duration_minutes"].between(
            config.min_duration_minutes, config.max_duration_minutes, inclusive="both"
        )
        & frame[PASSENGERS].between(
            config.min_passenger_count, config.max_passenger_count, inclusive="both"
        )
        & frame[PULOCATION].between(config.valid_zone_min, config.valid_zone_max)
        & frame[DOLOCATION].between(config.valid_zone_min, config.valid_zone_max)
    )
    if config.drop_exact_duplicates:
        valid &= ~duplicates
    cleaned = frame.loc[valid].copy()
    cleaned.sort_values(by=PICKUP, inplace=True, kind="stable")
    cleaned.reset_index(drop=True, inplace=True)
    input_rows = len(frame)
    rejected = input_rows - len(cleaned)
    report = ValidationReport(
        input_rows=input_rows,
        valid_rows=len(cleaned),
        rejected_rows=rejected,
        rejection_rate=rejected / input_rows if input_rows else 0.0,
        out_of_period_rows=int((frame[PICKUP].notna() & ~in_period).sum()),
        duplicate_rows=int(duplicates.sum()),
        period=str(period) if period is not None else None,
    )
    if cleaned.empty:
        raise ValueError("no valid rows remain after validation")
    return cleaned, report

```

A.38 src/taxi_duration/features/_init_.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""Leakage-safe feature engineering."""

```

A.39 src/taxi_duration/features/builder.py

Train ve serve için tek leakage-safe scikit-learn transformer.

```

from __future__ import annotations

from typing import Final

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

from taxi_duration.data.validation import PASSENGERS, PICKUP

INFERENCE_COLUMNS: Final = (PICKUP, "PULocationID", "DOLocationID", PASSENGERS)
# Zone identifiers are nominal: TLC LocationIDs follow the alphabetical order of zone names,
# so their numeric order carries no geography. They are encoded downstream (models.factory).
ZONE_COLUMNS: Final = ("route_id", "pickup_zone_id", "dropoff_zone_id")
ROUTE_MULTIPLIER: Final = 1000
FEATURE_COLUMNS: Final = (
    "route_id",
    "pickup_zone_id",
    "dropoff_zone_id",
    "passenger_count",
    "hour_sin",
    "hour_cos",
    "weekday_sin",
    "weekday_cos",
    "is_weekend",
)

```

```

        "is_rush_hour",
    )

class TripFeatureBuilder(BaseEstimator, TransformerMixin):
    """Deterministic features using only fields available at trip start."""

    def fit(self, X: pd.DataFrame, y: object = None) -> TripFeatureBuilder:
        self.feature_names_in_ = np.asarray(INFERENCE_COLUMNS, dtype=object)
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        missing = sorted(set(INFERENCE_COLUMNS) - set(X.columns))
        if missing:
            raise ValueError(f"missing inference columns: {missing}")
        pickup = pd.to_datetime(X[PICKUP], errors="raise")
        hour = pickup.dt.hour + pickup.dt.minute / 60.0
        weekday = pickup.dt.dayofweek
        result = pd.DataFrame(index=X.index)
        pickup_zone = pd.to_numeric(X[PULocationID], errors="raise").astype("int64")
        dropoff_zone = pd.to_numeric(X[DOLocationID], errors="raise").astype("int64")
        result["route_id"] = pickup_zone * ROUTE_MULTIPLIER + dropoff_zone
        result["pickup_zone_id"] = pickup_zone
        result["dropoff_zone_id"] = dropoff_zone
        result["passenger_count"] = pd.to_numeric(X[PASSENGERS], errors="raise")
        result["hour_sin"] = np.sin(2 * np.pi * hour / 24)
        result["hour_cos"] = np.cos(2 * np.pi * hour / 24)
        result["weekday_sin"] = np.sin(2 * np.pi * weekday / 7)
        result["weekday_cos"] = np.cos(2 * np.pi * weekday / 7)
        result["is_weekend"] = (weekday >= 5).astype(int)
        result["is_rush_hour"] = ((hour.between(7, 10)) | (hour.between(16, 20))).astype(int)
        return result[list(FEATURE_COLUMNS)]

    def get_feature_names_out(self, input_features: object = None) -> np.ndarray:
        return np.asarray(FEATURE_COLUMNS, dtype=object)

```

A.40 src/taxi_duration/logging.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from __future__ import annotations

import logging
import sys

import structlog

def configure_logging(level: str = "INFO") -> None:
    logging.basicConfig(format="%(message)s", stream=sys.stdout, level=level.upper())
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso", utc=True),
            structlog.processors.dict_tracebacks,
            structlog.processors.JSONRenderer(),
        ],
        wrapper_class=structlog.make_filtering_bound_logger(
            getattr(logging, level.upper(), logging.INFO)
        ),
        logger_factory=structlog.stdlib.LoggerFactory(),
        cache_logger_on_first_use=True,
    )

```

A.41 src/taxi_duration/models/_init_.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""Model construction and artifact I/O."""

```

A.42 src/taxi_duration/models/artifact.py

Model/metadata save-load ve lineage record.

```

from __future__ import annotations

import hashlib
import json
import platform
from dataclasses import asdict, dataclass
from datetime import UTC, datetime
from pathlib import Path
from typing import Any

import joblib
import numpy as np
import pandas as pd
import sklearn
from sklearn.pipeline import Pipeline

```

```

FEATURE_CONTRACT_VERSION = "1.2.0"

class ArtifactCompatibilityError(RuntimeError):
    """The serialized model was produced by a different library runtime than the loader."""

def runtime_versions() -> dict[str, str]:
    """Libraries whose version determines whether a joblib/pickle artifact loads correctly."""
    return {
        "python": platform.python_version(),
        "scikit-learn": sklearn.__version__,
        "numpy": np.__version__,
        "pandas": pd.__version__,
        "joblib": joblib.__version__,
    }

def check_runtime_compatibility(metadata: dict[str, Any]) -> None:
    """Refuse to load an artifact whose scikit-learn/numpy versions differ from this runtime.

    Unpickling across scikit-learn versions is unsupported and can fail or, worse, load and
    predict differently. Python patch versions are allowed to differ.
    """
    recorded = metadata.get("runtime_versions")
    if not isinstance(recorded, dict):
        raise ArtifactCompatibilityError("artifact metadata has no runtime_versions")
    current = runtime_versions()
    mismatches = [
        f"{name}: artifact={recorded.get(name)} runtime={current[name]}"
        for name in ("scikit-learn", "numpy", "pandas", "joblib")
        if recorded.get(name) != current[name]
    ]
    if recorded.get("python", "").rsplit(".", 1)[0] != current["python"].rsplit(".", 1)[0]:
        mismatches.append(f"python: artifact={recorded.get('python')} runtime={current['python']}")
    if mismatches:
        raise ArtifactCompatibilityError("incompatible runtime: " + "; ".join(mismatches))

@dataclass(frozen=True)
class ModelMetadata:
    model_version: str
    created_at_utc: str
    git_sha: str
    data_sha256: str
    config_sha256: str
    metrics: dict[str, float]
    runtime_versions: dict[str, str]
    feature_contract_version: str = FEATURE_CONTRACT_VERSION

def hash_text(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

def save_artifact(
    model: Pipeline,
    metadata: ModelMetadata,
    model_path: Path,
    metadata_path: Path,
) -> None:
    model_path.parent.mkdir(parents=True, exist_ok=True)
    metadata_path.parent.mkdir(parents=True, exist_ok=True)
    temporary = model_path.with_suffix(".joblib.tmp")
    joblib.dump(model, temporary)
    temporary.replace(model_path)
    metadata_path.write_text(
        json.dumps(asdict(metadata), indent=2, sort_keys=True), encoding="utf-8"
    )

def load_artifact(model_path: Path, metadata_path: Path) -> tuple[Pipeline, dict[str, Any]]:
    """Load a trusted artifact after verifying it matches the current library runtime."""
    metadata: dict[str, Any] = json.loads(metadata_path.read_text(encoding="utf-8"))
    check_runtime_compatibility(metadata)
    model = joblib.load(model_path)
    return model, metadata

def new_metadata(
    model_version: str,
    git_sha: str,
    data_sha256: str,
    config_sha256: str,
    metrics: dict[str, float],
) -> ModelMetadata:
    return ModelMetadata(
        model_version=model_version,
        created_at_utc=datetime.now(UTC).isoformat(),
        git_sha=git_sha,
        data_sha256=data_sha256,
        config_sha256=config_sha256,
        metrics=metrics,
    )

```

```

        runtime_versions=runtime_versions(),
    )

```

A.43 src/taxi_duration/models/factory.py

HistGradientBoosting ve log target wrapper içeren fitted Pipeline factory.

```

from __future__ import annotations

from typing import Any

import numpy as np
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import TargetEncoder

from taxi_duration.config import AppConfig
from taxi_duration.features.builder import FEATURE_COLUMNS, ZONE_COLUMNS, TripFeatureBuilder

def _zone_encoder(config: AppConfig) -> ColumnTransformer:
    """Encode nominal zone columns.

    ``target``: cross-fitted target encoding (out-of-fold during fit, full-train at inference)
    of the PU-D0 route and of each zone. Unseen routes fall back to the training target mean.
    Because the encoder sits inside TransformedTargetRegressor it encodes loglp(duration).

    ``ordinal``: v1.0 behaviour, raw IDs passed through as numbers. Kept as a comparison arm.
    """
    zone_columns = list(ZONE_COLUMNS)
    numeric_columns = [c for c in FEATURE_COLUMNS if c not in ZONE_COLUMNS]
    if config.model.zone_encoding == "target":
        zone_step: Any = TargetEncoder(
            target_type="continuous",
            smooth="auto",
            cv=config.model.target_encoder_cv,
            shuffle=True,
            random_state=config.random_seed,
        )
    else:
        zone_step = "passthrough"
    return ColumnTransformer(
        [("zones", zone_step, zone_columns), ("numeric", "passthrough", numeric_columns)],
        remainder="drop",
        verbose_feature_names_out=False,
    )

def build_model(config: AppConfig, max_iter: int | None = None) -> Pipeline:
    regressor = HistGradientBoostingRegressor(
        loss="absolute_error",
        learning_rate=config.model.learning_rate,
        max_iter=max_iter if max_iter is not None else config.model.max_iter,
        max_leaf_nodes=config.model.max_leaf_nodes,
        l2_regularization=config.model.l2_regularization,
        min_samples_leaf=config.model.min_samples_leaf,
        random_state=config.random_seed,
        # The number of boosting iterations is chosen on the chronological validation segment
        # (training.pipeline.select_iterations), not on an internal random holdout.
        early_stopping=False,
    )
    inner = Pipeline([("encoder", _zone_encoder(config)), ("hgb", regressor)])
    target_model = TransformedTargetRegressor(
        regressor=inner,
        func=np.loglp,
        inverse_func=np.expml,
        check_inverse=False,
    )
    return Pipeline([("features", TripFeatureBuilder()), ("regressor", target_model)])

def seen_routes(model: Pipeline) -> set[int] | None:
    """Routes observed by the fitted target encoder, or None when not target-encoded."""
    try:
        encoder = model.named_steps["regressor"].regressor_.named_steps["encoder"]
        zones = encoder.named_transformers_["zones"]
    except (AttributeError, KeyError):
        return None
    if not isinstance(zones, TargetEncoder):
        return None
    return {int(v) for v in zones.categories_[0]}

```

A.44 src/taxi_duration/models/predict.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from __future__ import annotations

from typing import cast

```

```

import numpy as np
import pandas as pd
from numpy.typing import NDArray
from sklearn.pipeline import Pipeline

from taxi_duration.config import ServingConfig

def predict_duration(
    model: Pipeline, features: pd.DataFrame, config: ServingConfig
) -> NDArray[np.float64]:
    """Apply the single train/evaluate/serve prediction contract."""
    raw = np.asarray(model.predict(features), dtype=float)
    return cast(
        NDArray[np.float64],
        np.clip(raw, config.prediction_lower_bound, config.prediction_upper_bound),
    )

```

A.45 src/taxi_duration/monitoring/__init__.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""Operational and statistical monitoring."""

```

A.46 src/taxi_duration/monitoring/drift.py

Current batch'i reference ile karşılaştıran drift report.

```

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline

from taxi_duration.config import MonitoringConfig, ServingConfig
from taxi_duration.data.validation import DOLOCATION, PASSENGERS, PICKUP, PULOCATION
from taxi_duration.features.builder import INFERENCE_COLUMNS
from taxi_duration.models.predict import predict_duration
from taxi_duration.monitoring.profile import population_stability_index

def _profile_values(
    values: np.ndarray, edges: list[float]
) -> tuple[list[float], dict[str, float | int]]:
    """Split a feature into (histogram of valid values, data-quality counts).

    np.histogram silently drops NaN and out-of-range values and the proportions would then be
    renormalised over the survivors, so a batch where a large share of rows is broken would
    still look distribution-stable. Invalid values are therefore counted and reported apart.
    """
    numeric = pd.to_numeric(pd.Series(values), errors="coerce").to_numpy(dtype=float)
    missing = np.isnan(numeric)
    out_of_range = ~missing & ((numeric < edges[0]) | (numeric > edges[-1]))
    valid = numeric[~missing & ~out_of_range]
    counts, _ = np.histogram(valid, bins=np.asarray(edges))
    total = len(numeric)
    quality: dict[str, float | int] = {
        "rows": total,
        "valid_rows": len(valid),
        "missing_rows": int(missing.sum()),
        "out_of_range_rows": int(out_of_range.sum()),
        "invalid_rate": float((missing.sum() + out_of_range.sum()) / total) if total else 0.0,
    }
    return (counts / max(counts.sum(), 1)).tolist(), quality

def _status(psi: float, invalid_rate: float, config: MonitoringConfig) -> str:
    if psi >= config.psi_critical or invalid_rate > config.max_invalid_rate:
        return "critical"
    if psi >= config.psi_warning:
        return "warning"
    return "ok"

def _categorical_psi(values: pd.Series, expected: dict[str, float]) -> float:
    normalized = values.astype(str).value_counts(normalize=True)
    categories = [name for name in expected if name != "__OTHER__"]
    actual = [float(normalized.get(name, 0.0)) for name in categories]
    actual.append(max(0.0, 1.0 - sum(actual)))
    expected_values = [float(expected[name]) for name in categories]
    expected_values.append(float(expected.get("__OTHER__", 0.0)))
    return population_stability_index(expected_values, actual)

def _valid_inference_rows(current: pd.DataFrame) -> pd.DataFrame:
    frame = current.loc[:, list(INFERENCE_COLUMNS)].copy()
    frame[PICKUP] = pd.to_datetime(frame[PICKUP], errors="coerce")

```

```

for name in (PULOCATION, DOLOCATION, PASSENGERS):
    frame[name] = pd.to_numeric(frame[name], errors="coerce")
return frame.dropna()

def drift_report(
    current: pd.DataFrame,
    reference_path: Path,
    config: MonitoringConfig,
    *,
    model: Pipeline | None = None,
    serving: ServingConfig | None = None,
) -> dict[str, Any]:
    reference = json.loads(reference_path.read_text(encoding="utf-8"))
    pickup = pd.to_datetime(current[PICKUP], errors="coerce")
    features = {
        "pickup_hour": pickup.dt.hour.to_numpy(dtype=float, na_value=np.nan),
        "passenger_count": current[PASSENGERS].to_numpy(dtype=float, na_value=np.nan),
    }
    report: dict[str, Any] = {"row_count": len(current), "features": {}}
    for name, values in features.items():
        expected = reference[name]
        actual, quality = _profile_values(values, expected["edges"])
        psi = population_stability_index(expected["proportions"], actual)
        report["features"][name] = {
            "psi": psi,
            "data_quality": quality,
            "status": _status(psi, float(quality["invalid_rate"]), config),
        }
    valid = _valid_inference_rows(current)
    categorical_values = {
        "pickup_zone": valid[PULOCATION].astype(int).astype(str),
        "dropoff_zone": valid[DOLOCATION].astype(int).astype(str),
        "route": valid[PULOCATION].astype(int).astype(str)
        + ">"
        + valid[DOLOCATION].astype(int).astype(str),
    }
    categorical_report: dict[str, Any] = {}
    for name, categorical_series in categorical_values.items():
        psi = _categorical_psi(categorical_series, reference["categorical"][name])
        categorical_report[name] = {"psi": psi, "status": _status(psi, 0.0, config)}
    known_routes = set(reference["known_routes"])
    route_values = categorical_values["route"]
    unseen_rate = float((~route_values.isin(known_routes)).mean()) if len(route_values) else 1.0
    categorical_report["route"]["unseen_rate"] = unseen_rate
    if unseen_rate > config.max_unseen_route_rate:
        categorical_report["route"]["status"] = "critical"
    report["categorical"] = categorical_report

    if model is not None and serving is not None and "prediction_minutes" in reference:
        prediction = predict_duration(model, valid, serving)
        expected_prediction = reference["prediction_minutes"]
        actual, quality = _profile_values(prediction, expected_prediction["edges"])
        psi = population_stability_index(expected_prediction["proportions"], actual)
        report["prediction_minutes"] = {
            "psi": psi,
            "data_quality": quality,
            "status": _status(psi, float(quality["invalid_rate"]), config),
        }
    statuses = [f["status"] for f in report["features"].values()]
    statuses.extend(f["status"] for f in categorical_report.values())
    if "prediction_minutes" in report:
        statuses.append(report["prediction_minutes"]["status"])
    report["status"] = (
        "critical" if "critical" in statuses else "warning" if "warning" in statuses else "ok"
    )
    return report

```

A.47 src/taxi_duration/monitoring/profile.py

Training reference histogramları ve PSI primitive'i.

```

from __future__ import annotations

import json
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline

from taxi_duration.config import ServingConfig
from taxi_duration.data.validation import DOLOCATION, PASSENGERS, PICKUP, PULOCATION
from taxi_duration.features.builder import INFERENCE_COLUMNS
from taxi_duration.models.predict import predict_duration

def _histogram(values: np.ndarray, bins: np.ndarray) -> dict[str, list[float]]:
    counts, edges = np.histogram(values, bins=bins)
    proportions = counts / max(counts.sum(), 1)
    return {"edges": edges.tolist(), "proportions": proportions.tolist()}

```

```

def _categorical_profile(values: pd.Series, top_n: int = 50) -> dict[str, float]:
    normalized = values.astype(str).value_counts(normalize=True)
    top = normalized.head(top_n)
    result = {str(key): float(value) for key, value in top.items()}
    result["__OTHER__"] = max(0.0, 1.0 - sum(result.values()))
    return result

def _routes(frame: pd.DataFrame) -> pd.Series:
    return (
        frame[PULLOCATION].astype(int).astype(str) + ">" + frame[DOLLOCATION].astype(int).astype(str)
    )

def build_reference_profile(
    frame: pd.DataFrame,
    output: Path,
    *,
    model: Pipeline | None = None,
    serving: ServingConfig | None = None,
) -> dict[str, object]:
    pickup = pd.to_datetime(frame[PICKUP])
    profile: dict[str, object] = {
        "row_count": len(frame),
        "pickup_hour": _histogram(pickup.dt.hour.to_numpy(), np.arange(0, 25, 2)),
        "passenger_count": _histogram(frame[PASSENGERS].to_numpy(), np.arange(0.5, 7.5, 1)),
        "duration_minutes": _histogram(
            frame["duration_minutes"].to_numpy(), np.linspace(1, 120, 13)
        ),
        "categorical": {
            "pickup_zone": _categorical_profile(frame[PULLOCATION]),
            "dropoff_zone": _categorical_profile(frame[DOLLOCATION]),
            "route": _categorical_profile(_routes(frame), top_n=100),
        },
        "known_routes": sorted(_routes(frame).unique().tolist()),
    }
    if model is not None and serving is not None:
        prediction = predict_duration(model, frame.loc[:, list(INFERENCE_COLUMNS)], serving)
        profile["prediction_minutes"] = _histogram(
            prediction,
            np.linspace(serving.prediction_lower_bound, serving.prediction_upper_bound, 25),
        )
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(profile, indent=2, sort_keys=True), encoding="utf-8")
    return profile

def population_stability_index(expected: list[float], actual: list[float]) -> float:
    expected_arr = np.clip(np.asarray(expected, dtype=float), 1e-6, None)
    actual_arr = np.clip(np.asarray(actual, dtype=float), 1e-6, None)
    if expected_arr.shape != actual_arr.shape:
        raise ValueError("expected and actual histograms must have the same number of bins")
    return float(np.sum((actual_arr - expected_arr) * np.log(actual_arr / expected_arr)))

```

A.48 src/taxi_duration/serving/_init_.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

"""FastAPI inference service."""

```

A.49 src/taxi_duration/serving/app.py

Startup loading, endpoints, timezone conversion, prediction ve error semantics.

```

from __future__ import annotations

import os
import time
import uuid
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Any, cast
from zoneinfo import ZoneInfo

import pandas as pd
import structlog
from fastapi import FastAPI, HTTPException, Request
from fastapi.exception_handlers import request_validation_exception_handler
from fastapi.exceptions import RequestValidationError
from fastapi.responses import Response
from prometheus_client import CONTENT_TYPE_LATEST, generate_latest
from sklearn.pipeline import Pipeline

from taxi_duration.config import AppConfig, load_config
from taxi_duration.logging import configure_logging
from taxi_duration.models.artifact import load_artifact
from taxi_duration.models.predict import predict_duration
from taxi_duration.serving.metrics import LATENCY, PREDICTION, REQUESTS
from taxi_duration.serving.schemas import HealthResponse, PredictionRequest, PredictionResponse

```

```

configure_logging(os.getenv("TAXI_LOG_LEVEL", "INFO"))
LOGGER = structlog.get_logger(__name__)

@asynccontextmanager
async def lifespan(application: FastAPI) -> AsyncIterator[None]:
    config = load_config(os.getenv("TAXI_CONFIG_PATH", "configs/development.yaml"))
    model_path = Path(os.getenv("TAXI_MODEL_PATH", "artifacts/model.joblib"))
    metadata_path = Path(os.getenv("TAXI_METADATA_PATH", "artifacts/metadata.json"))
    try:
        model, metadata = load_artifact(model_path, metadata_path)
        application.state.model = model
        application.state.metadata = metadata
        application.state.config = config
        application.state.ready = True
        LOGGER.info("model_loaded", model_version=metadata["model_version"])
    except Exception as exc:
        # Any load failure (missing file, runtime mismatch, corrupt pickle) keeps the process
        # alive but not ready, so the orchestrator withholds traffic instead of crash-looping.
        application.state.ready = False
        application.state.load_error = f"{type(exc).__name__}: {exc}"
        LOGGER.error("model_load_failed", error=application.state.load_error)
    yield

app = FastAPI(
    title="NYC Taxi Duration API",
    version="1.2.3",
    docs_url="/docs",
    redoc_url=None,
    lifespan=lifespan,
)

@app.exception_handler(RequestValidationError)
async def count_contract_violations(request: Request, exc: RequestValidationError) -> Response:
    """Count 422s so the error-rate panel also sees contract violations, then use the default."""
    if request.url.path == "/v1/predict":
        REQUESTS.labels(status="invalid").inc()
    return await request_validation_exception_handler(request, exc)

@app.middleware("http")
async def request_context(request: Request, call_next: Any) -> Response:
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    structlog.contextvars.bind_contextvars(request_id=request_id)
    started = time.perf_counter()
    status_code = 500
    try:
        response = cast(Response, await call_next(request))
        status_code = response.status_code
        return response
    finally:
        elapsed = time.perf_counter() - started
        LOGGER.info(
            "request_completed",
            path=request.url.path,
            status_code=status_code,
            latency_seconds=elapsed,
        )
    structlog.contextvars.clear_contextvars()

@app.get("/health/live", response_model=HealthResponse)
def live() -> HealthResponse:
    return HealthResponse(status="alive")

@app.get("/health/ready", response_model=HealthResponse)
def ready(request: Request) -> HealthResponse:
    if not getattr(request.app.state, "ready", False):
        raise HTTPException(status_code=503, detail="model unavailable")
    return HealthResponse(
        status="ready", model_version=str(request.app.state.metadata["model_version"])
    )

@app.post("/v1/predict", response_model=PredictionResponse)
def predict(payload: PredictionRequest, request: Request) -> PredictionResponse:
    if not getattr(request.app.state, "ready", False):
        REQUESTS.labels(status="unavailable").inc()
        raise HTTPException(status_code=503, detail="model unavailable")
    started = time.perf_counter()
    try:
        config: AppConfig = request.app.state.config
        local_time = payload.pickup_datetime.astimezone(ZoneInfo(config.timezone)).replace(
            tzinfo=None
        )
        features = pd.DataFrame(
            [
                {
                    "tpep_pickup_datetime": local_time,
                    "PULocationID": payload.pickup_location_id,

```

```

        "DOLocationID": payload.dropoff_location_id,
        "passenger_count": payload.passenger_count,
    }
}
)
model: Pipeline = request.app.state.model
prediction = float(predict_duration(model, features, config.serving)[0])
REQUESTS.labels(status="success").inc()
PREDICTION.observe(prediction)
metadata = request.app.state.metadata
LOGGER.info(
    "prediction_completed",
    prediction_minutes=prediction,
    model_version=metadata["model_version"],
    pickup_location_id=payload.pickup_location_id,
    dropoff_location_id=payload.dropoff_location_id,
)
return PredictionResponse(
    predicted_duration_minutes=round(prediction, 2),
    model_version=str(metadata["model_version"]),
    feature_contract_version=str(metadata["feature_contract_version"]),
)
except Exception:
    REQUESTS.labels(status="error").inc()
    LOGGER.exception("prediction_failed")
    raise HTTPException(status_code=500, detail="prediction failed") from None
finally:
    LATENCY.observe(time.perf_counter() - started)

@app.get("/metrics", include_in_schema=False)
def metrics() -> Response:
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

```

A.50 src/taxi_duration/serving/metrics.py

Prometheus metric definitions.

```

from prometheus_client import Counter, Histogram

REQUESTS = Counter("taxi_prediction_requests_total", "Prediction requests", labelnames=("status",))
LATENCY = Histogram(
    "taxi_prediction_latency_seconds",
    "Prediction endpoint latency",
    buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0),
)
PREDICTION = Histogram(
    "taxi_predicted_duration_minutes",
    "Predicted trip duration",
    buckets=(1, 5, 10, 15, 20, 30, 45, 60, 90, 120),
)

```

A.51 src/taxi_duration/serving/schemas.py

Version 1 HTTP input-output contract.

```

from __future__ import annotations

from datetime import datetime

from pydantic import BaseModel, ConfigDict, Field, field_validator

class PredictionRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    pickup_datetime: datetime
    # TLC zones 264 (Unknown) and 265 (Outside of NYC) are not routable destinations.
    pickup_location_id: int = Field(ge=1, le=263)
    dropoff_location_id: int = Field(ge=1, le=263)
    passenger_count: int = Field(ge=1, le=6)

    @field_validator("pickup_datetime")
    @classmethod
    def require_timezone(cls, value: datetime) -> datetime:
        if value.tzinfo is None or value.utcoffset() is None:
            raise ValueError("pickup_datetime must include a UTC offset")
        return value

class PredictionResponse(BaseModel):
    predicted_duration_minutes: float
    model_version: str
    feature_contract_version: str

class HealthResponse(BaseModel):
    status: str
    model_version: str | None = None

```

A.52 src/taxi_duration/training/__init__.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
"""Training, evaluation, and release gates."""
```

A.53 src/taxi_duration/training/evaluate.py

Regression metrics ve tail/bias ölçümü.

```
from __future__ import annotations

from typing import Any

import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from taxi_duration.data.validation import DOLOCATION, PICKUP, PULOCATION
from taxi_duration.features.builder import ROUTE_MULTIPLIER

def regression_metrics(y_true: pd.Series, y_pred: np.ndarray) -> dict[str, float]:
    """Point-estimate metrics.

    The model estimates the conditional *median* (L1 loss; quantiles are invariant under the
    monotone loglp transform), so for right-skewed durations bias_minutes is expected to be
    negative. It is reported for monitoring, not gated.
    """
    absolute_error = np.abs(y_true.to_numpy() - y_pred)
    return {
        "mae_minutes": float(mean_absolute_error(y_true, y_pred)),
        "rmse_minutes": float(mean_squared_error(y_true, y_pred) ** 0.5),
        "r2": float(r2_score(y_true, y_pred)),
        "p95_absolute_error_minutes": float(np.quantile(absolute_error, 0.95)),
        "bias_minutes": float(np.mean(y_pred - y_true.to_numpy())),
    }

def _slice_metrics(frame: pd.DataFrame, key: pd.Series) -> dict[str, dict[str, float]]:
    out: dict[str, dict[str, float]] = {}
    for value, group in frame.groupby(key, observed=True, sort=True):
        error = group["prediction"] - group["actual"]
        out[str(value)] = {
            "rows": len(group),
            "mae_minutes": float(error.abs().mean()),
            "bias_minutes": float(error.mean()),
        }
    return out

def error_breakdown(
    features: pd.DataFrame,
    y_true: pd.Series,
    y_pred: np.ndarray,
    seen_routes: set[int] | None = None,
) -> dict[str, Any]:
    """MAE and bias by pickup hour, weekday and (when available) route coverage."""
    frame = pd.DataFrame({"actual": y_true.to_numpy(), "prediction": y_pred}, index=features.index)
    pickup = pd.to_datetime(features[PICKUP])
    report: dict[str, Any] = {
        "by_pickup_hour": _slice_metrics(frame, pickup.dt.hour.rename("hour")),
        "by_weekday": _slice_metrics(frame, pickup.dt.dayofweek.rename("weekday")),
    }
    if seen_routes is not None:
        route = features[PULOCATION].astype("int64") * ROUTE_MULTIPLIER + features[
            DOLOCATION
        ].astype("int64")
        seen = route.isin(seen_routes).map({True: "seen_in_training", False: "unseen_route"})
        report["by_route_coverage"] = _slice_metrics(frame, seen.rename("route_coverage"))
    return report
```

A.54 src/taxi_duration/training/gates.py

Promotion kararını açıklayıcı failure listesiyle üretir.

```
from __future__ import annotations

from collections.abc import Mapping
from dataclasses import dataclass
from statistics import NormalDist

import numpy as np
import pandas as pd

from taxi_duration.config import GateConfig

@dataclass(frozen=True)
class PairedDelta:
```

```

mean: float
standard_error: float
upper_confidence: float
clusters: int

def paired_delta_upper_bound(
    delta: np.ndarray, clusters: np.ndarray, confidence_level: float
) -> PairedDelta:
    """One-sided upper confidence bound of mean(delta) with cluster-robust standard error.

    delta = |y - candidate| - |y - champion| per test row (positive = candidate worse).
    Rows in the same cluster (pickup hour) share traffic conditions, so their deltas are
    correlated; an i.i.d. standard error would overstate the evidence. The cluster-robust
    variance of the mean is  $G/(G-1) * \sum_g (\sum_{i \in g} (d_i - \bar{d})^2) / n^2$ .
    """
    values = np.asarray(delta, dtype=float)
    n = len(values)
    if n < 2:
        raise ValueError("at least two paired rows are required")
    mean = float(values.mean())
    residual_sums = pd.Series(values - mean).groupby(np.asarray(clusters)).sum().to_numpy()
    groups = len(residual_sums)
    if groups < 2:
        raise ValueError("at least two clusters are required")
    variance = groups / (groups - 1) * float(np.sum(residual_sums**2)) / n**2
    standard_error = float(np.sqrt(variance))
    z_score = NormalDist().inv_cdf(confidence_level)
    return PairedDelta(mean, standard_error, mean + z_score * standard_error, groups)

@dataclass(frozen=True)
class GateResult:
    passed: bool
    failures: tuple[str, ...]

@dataclass(frozen=True)
class ChampionComparison:
    """Outcome of scoring the current production model on the candidate's test window.

    status: "compared" (champion_test_mae is set), "skipped" (no champion supplied, e.g. the
    first release) or "failed" (a champion was supplied but could not be evaluated).
    """
    status: str
    champion_test_mae: float | None = None
    champion_version: str | None = None
    candidate_test_mae: float | None = None
    mae_delta: float | None = None
    mae_delta_upper_confidence: float | None = None
    mae_delta_standard_error: float | None = None
    noninferiority_margin_minutes: float | None = None
    clusters: int | None = None
    confidence_level: float | None = None
    detail: str | None = None

def evaluate_release_gates(
    validation: dict[str, float],
    test: dict[str, float],
    baseline_validation_mae: float,
    config: GateConfig,
    champion: ChampionComparison | None = None,
    data_quality: Mapping[str, float | int] | None = None,
) -> GateResult:
    failures: list[str] = []
    if validation["mae_minutes"] > config.max_validation_mae_minutes:
        failures.append("validation MAE exceeds limit")
    if test["mae_minutes"] > config.max_test_mae_minutes:
        failures.append("test MAE exceeds limit")
    if test["p95_absolute_error_minutes"] > config.max_p95_absolute_error_minutes:
        failures.append("test p95 absolute error exceeds limit")
    improvement = 1 - validation["mae_minutes"] / baseline_validation_mae
    if improvement < config.min_improvement_vs_baseline:
        failures.append("candidate does not improve sufficiently over median baseline")
    if abs(test["mae_minutes"] - validation["mae_minutes"]) > config.max_validation_test_gap:
        failures.append("validation-to-test MAE gap exceeds limit")
    if champion is not None:
        if champion.status == "failed":
            failures.append(f"champion could not be evaluated: {champion.detail}")
        elif champion.status == "compared":
            # Non-inferiority: the candidate must show, at the configured confidence, that it is
            # not worse than the champion by more than the margin. A margin of zero would turn
            # this into a superiority test that blocks ~95% of equally good retrains.
            if champion.champion_test_mae is None or champion.mae_delta_upper_confidence is None:
                failures.append("champion comparison is incomplete")
            else:
                margin = champion.champion_test_mae * config.max_regression_vs_champion
                if champion.mae_delta_upper_confidence > margin:
                    failures.append(
                        "candidate is not shown to be non-inferior to the production champion"
                    )

```

```

if data_quality is not None:
    if float(data_quality["rejection_rate"]) > config.max_rejection_rate:
        failures.append("data rejection rate exceeds limit")
    if float(data_quality["out_of_period_rate"]) > config.max_out_of_period_rate:
        failures.append("out-of-period row rate exceeds limit")
    if float(data_quality["duplicate_rate"]) > config.max_duplicate_rate:
        failures.append("duplicate row rate exceeds limit")
    if int(data_quality["valid_rows"]) < config.min_valid_rows:
        failures.append("valid row count is below minimum")
return GateResult(passed=not failures, failures=tuple(failures))

```

A.55 src/taxi_duration/training/pipeline.py

Offline lifecycle orchestrator: validate, split, fit, evaluate, gate, package, profile.

```

from __future__ import annotations

import json
import os
from collections.abc import Sequence
from dataclasses import asdict
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
import structlog
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.pipeline import Pipeline

from taxi_duration.config import AppConfig, canonical_config_json
from taxi_duration.data.ingest import sha256_file
from taxi_duration.data.validation import PICKUP, Period, validate_and_clean
from taxi_duration.features.builder import INFERENCE_COLUMNS
from taxi_duration.models.artifact import hash_text, load_artifact, new_metadata, save_artifact
from taxi_duration.models.factory import build_model, seen_routes
from taxi_duration.models.predict import predict_duration
from taxi_duration.monitoring.profile import build_reference_profile
from taxi_duration.training.evaluate import error_breakdown, regression_metrics
from taxi_duration.training.gates import (
    ChampionComparison,
    evaluate_release_gates,
    paired_delta_upper_bound,
)
from taxi_duration.training.split import chronological_split

LOGGER = structlog.get_logger(__name__)
TARGET = "duration_minutes"

def _git_sha() -> str:
    explicit = os.getenv("GIT_SHA")
    if explicit:
        return explicit
    head_path = Path(".git/HEAD")
    if not head_path.exists():
        return "unknown"
    head = head_path.read_text(encoding="utf-8").strip()
    if head.startswith("ref: "):
        reference = Path(".git") / head.removeprefix("ref: ")
        return reference.read_text(encoding="utf-8").strip() if reference.exists() else "unknown"
    return head

def select_iterations(
    model: Pipeline,
    X_validation: pd.DataFrame,
    y_validation: pd.Series,
    config: AppConfig,
) -> tuple[int, list[float]]:
    """Pick the boosting iteration count that minimises MAE on the chronological validation set.

    Replaces HistGradientBoosting's built-in early stopping, which holds out a *random* 10% of
    the training rows and therefore mixes time periods inside a chronological design.
    """
    target_model = model.named_steps["regressor"]
    inner = target_model.regressor_
    transformed = inner[:-1].transform(model.named_steps["features"].transform(X_validation))
    curve = [
        float(
            mean_absolute_error(
                y_validation,
                np.clip(
                    target_model.inverse_func(stage),
                    config.serving.prediction_lower_bound,
                    config.serving.prediction_upper_bound,
                ),
            )
        )
        for stage in inner.named_steps["hgb"].staged_predict(transformed)
    ]

```

```

return int(np.argmin(curve)) + 1, curve

def evaluate_champion(
    model_path: Path | None,
    metadata_path: Path | None,
    X_test: pd.DataFrame,
    y_test: pd.Series,
    candidate_prediction: np.ndarray,
    config: AppConfig,
) -> ChampionComparison:
    if model_path is None or metadata_path is None:
        return ChampionComparison(status="skipped", detail="no champion supplied")
    if not model_path.exists() or not metadata_path.exists():
        return ChampionComparison(status="skipped", detail="champion artifact not found")
    try:
        champion, metadata = load_artifact(model_path, metadata_path)
        prediction = predict_duration(champion, X_test, config.serving)
    except Exception as exc: # any failure must block promotion, not silently pass
        return ChampionComparison(status="failed", detail=f"type(exc).__name__: {exc}")
    actual = y_test.to_numpy(dtype=float)
    paired_delta = np.abs(actual - candidate_prediction) - np.abs(actual - prediction)
    hour_clusters = pd.to_datetime(X_test[PICKUP]).dt.floor("h").to_numpy()
    try:
        statistic = paired_delta_upper_bound(
            paired_delta, hour_clusters, config.gates.champion_confidence_level
        )
    except ValueError as exc:
        return ChampionComparison(status="failed", detail=f"comparison statistic: {exc}")
    champion_mae = float(mean_absolute_error(y_test, prediction))
    return ChampionComparison(
        status="compared",
        champion_test_mae=champion_mae,
        champion_version=str(metadata.get("model_version")),
        candidate_test_mae=float(mean_absolute_error(y_test, candidate_prediction)),
        mae_delta=statistic.mean,
        mae_delta_upper_confidence=statistic.upper_confidence,
        mae_delta_standard_error=statistic.standard_error,
        noninferiority_margin_minutes=champion_mae * config.gates.max_regression_vs_champion,
        clusters=statistic.clusters,
        confidence_level=config.gates.champion_confidence_level,
    )

def _quality_summary(reports: Sequence[dict[str, Any]]) -> dict[str, float | int]:
    input_rows = sum(int(report["input_rows"]) for report in reports)
    valid_rows = sum(int(report["valid_rows"]) for report in reports)
    rejected_rows = sum(int(report["rejected_rows"]) for report in reports)
    out_of_period_rows = sum(int(report["out_of_period_rows"]) for report in reports)
    duplicate_rows = sum(int(report["duplicate_rows"]) for report in reports)
    denominator = max(input_rows, 1)
    return {
        "input_rows": input_rows,
        "valid_rows": valid_rows,
        "rejected_rows": rejected_rows,
        "rejection_rate": rejected_rows / denominator,
        "out_of_period_rows": out_of_period_rows,
        "out_of_period_rate": out_of_period_rows / denominator,
        "duplicate_rows": duplicate_rows,
        "duplicate_rate": duplicate_rows / denominator,
    }

def train_from_parquet(
    data_path: Path | Sequence[Path],
    config: AppConfig,
    config_path: Path,
    model_path: Path,
    metadata_path: Path,
    metrics_path: Path,
    model_version: str,
    *,
    period: Period | None = None,
    champion_model_path: Path | None = None,
    champion_metadata_path: Path | None = None,
    reference_path: Path | None = None,
) -> dict[str, Any]:
    data_paths = [data_path] if isinstance(data_path, Path) else list(data_path)
    if not data_paths:
        raise ValueError("at least one training data path is required")
    if period is not None and len(data_paths) != 1:
        raise ValueError("an explicit period can only be used with one data file")
    frames: list[pd.DataFrame] = []
    reports: list[dict[str, Any]] = []
    periods: list[str | None] = []
    for path in data_paths:
        source_period = period if len(data_paths) == 1 else Period.from_filename(path)
        if len(data_paths) > 1 and source_period is None:
            raise ValueError(f"cannot infer period from rolling-window file: {path}")
        clean_frame, quality_report = validate_and_clean(
            pd.read_parquet(path), config.data, source_period
        )
        frames.append(clean_frame)

```

```

        quality_payload = asdict(quality_report)
        quality_payload["out_of_period_rate"] = quality_report.out_of_period_rate
        quality_payload["duplicate_rate"] = quality_report.duplicate_rate
        reports.append(quality_payload)
        periods.append(str(source_period) if source_period else None)
    clean = pd.concat(frames, ignore_index=True).sort_values(PICKUP, kind="stable")
    clean.reset_index(drop=True, inplace=True)
    quality = _quality_summary(reports)
    source_files = {path.name: sha256_file(path) for path in data_paths}
    split = chronological_split(clean, config.split)
    columns = list(INFERENCE_COLUMNS)
    X_train, y_train = split.train.loc[:, columns], split.train[TARGET]
    X_val, y_val = split.validation.loc[:, columns], split.validation[TARGET]
    X_test, y_test = split.test.loc[:, columns], split.test[TARGET]

    probe = build_model(config).fit(X_train, y_train)
    best_iter, curve = select_iterations(probe, X_val, y_val, config)
    model = (
        probe if best_iter == len(curve) else build_model(config, best_iter).fit(X_train, y_train)
    )

    validation_pred = predict_duration(model, X_val, config.serving)
    test_pred = predict_duration(model, X_test, config.serving)

    baseline = DummyRegressor(strategy="median")
    baseline.fit(np.zeros((len(split.train), 1)), y_train)
    baseline_pred = baseline.predict(np.zeros((len(split.validation), 1)))

    validation_metrics = regression_metrics(y_val, validation_pred)
    test_metrics = regression_metrics(y_test, test_pred)
    baseline_mae = regression_metrics(y_val, baseline_pred)["mae_minutes"]
    champion = evaluate_champion(
        champion_model_path, champion_metadata_path, X_test, y_test, test_pred, config
    )
    gate = evaluate_release_gates(
        validation_metrics, test_metrics, baseline_mae, config.gates, champion, quality
    )

    metrics: dict[str, Any] = {
        "data_quality": quality,
        "data_quality_by_file": reports,
        "source_periods": periods,
        "source_files_sha256": source_files,
        "rows": {
            "train": len(split.train),
            "validation": len(split.validation),
            "test": len(split.test),
        },
        "periods": {
            "train_start": str(split.train[PICKUP].min()),
            "train_end": str(split.train[PICKUP].max()),
            "validation_start": str(split.validation[PICKUP].min()),
            "validation_end": str(split.validation[PICKUP].max()),
            "test_start": str(split.test[PICKUP].min()),
            "test_end": str(split.test[PICKUP].max()),
        },
        "model": {
            "zone_encoding": config.model.zone_encoding,
            "selected_iterations": best_iter,
            "max_iterations": len(curve),
        },
        "baseline_validation_mae_minutes": baseline_mae,
        "validation": validation_metrics,
        "test": test_metrics,
        "test_breakdown": error_breakdown(X_test, y_test, test_pred, seen_routes(model)),
        "champion": asdict(champion),
        "release_gate": {"passed": gate.passed, "failures": list(gate.failures)},
    }
    metrics_path.parent.mkdir(parents=True, exist_ok=True)
    metrics_path.write_text(json.dumps(metrics, indent=2, sort_keys=True), encoding="utf-8")
    if not gate.passed:
        raise RuntimeError("release gates failed: " + " ".join(gate.failures))

    flat_metrics = {f"validation_{k}": v for k, v in validation_metrics.items()}
    flat_metrics.update({f"test_{k}": v for k, v in test_metrics.items()})
    metadata = new_metadata(
        model_version=model_version,
        git_sha=git_sha(),
        data_sha256=hash_text(json.dumps(source_files, sort_keys=True)),
        config_sha256=hash_text(canonical_config_json(config)),
        metrics=flat_metrics,
    )
    save_artifact(model, metadata, model_path, metadata_path)
    build_reference_profile(
        split.train,
        reference_path or config.monitoring.reference_path,
        model=model,
        serving=config.serving,
    )
    LOGGER.info("training_completed", model_version=model_version, metrics=metrics)
    return metrics

def evaluate_artifact(

```

```

data_path: Path,
config: AppConfig,
model_path: Path,
metadata_path: Path,
period: Period | None = None,
) -> dict[str, Any]:
    """Score a released artifact on a later source month (true out-of-period evaluation)."""
    period = period or Period.from_filename(data_path)
    model, metadata = load_artifact(model_path, metadata_path)
    clean, quality = validate_and_clean(pd.read_parquet(data_path), config.data, period)
    X, y = clean.loc[:, list(INFERENCE_COLUMNS)], clean[TARGET]
    prediction = predict_duration(model, X, config.serving)
    median_mae = float(np.abs(y - y.median()).mean())
    return {
        "model_version": metadata.get("model_version"),
        "data_quality": asdict(quality),
        "metrics": regression_metrics(y, prediction),
        "oracle_constant_median_mae_minutes": median_mae,
        "breakdown": error_breakdown(X, y, prediction, seen_routes(model)),
    }

```

A.56 src/taxi_duration/training/split.py

Chronological train/validation/test boundary.

```

from __future__ import annotations

from dataclasses import dataclass

import pandas as pd

from taxi_duration.config import SplitConfig

@dataclass(frozen=True)
class DataSplit:
    train: pd.DataFrame
    validation: pd.DataFrame
    test: pd.DataFrame

def chronological_split(frame: pd.DataFrame, config: SplitConfig) -> DataSplit:
    if len(frame) < 30:
        raise ValueError("at least 30 valid rows are required")
    train_end = int(len(frame) * config.train_fraction)
    validation_end = int(len(frame) * (config.train_fraction + config.validation_fraction))
    return DataSplit(
        train=frame.iloc[:train_end].copy(),
        validation=frame.iloc[train_end:validation_end].copy(),
        test=frame.iloc[validation_end:].copy(),
    )

```

A.57 tests/conftest.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from __future__ import annotations

from datetime import datetime

import numpy as np
import pandas as pd
import pytest

@pytest.fixture()
def synthetic_trips() -> pd.DataFrame:
    rng = np.random.default_rng(42)
    rows = 240
    pickup = pd.date_range("2025-01-01", periods=rows, freq="30min")
    pickup_zone = rng.integers(1, 50, rows)
    dropoff_zone = rng.integers(1, 50, rows)
    passenger = rng.integers(1, 5, rows)
    duration = 6 + np.abs(dropoff_zone - pickup_zone) * 0.45 + pickup.hour * 0.12
    duration += rng.normal(0, 1.0, rows)
    duration = np.clip(duration, 1.1, 60)
    return pd.DataFrame(
        {
            "tpep_pickup_datetime": pickup,
            "tpep_dropoff_datetime": pickup + pd.to_timedelta(duration, unit="m"),
            "PULocationID": pickup_zone,
            "DOLocationID": dropoff_zone,
            "passenger_count": passenger,
        }
    )

@pytest.fixture()
def aware_pickup() -> datetime:
    return datetime.fromisoformat("2025-01-15T08:30:00-05:00")

```

A.58 tests/contract/test_api_schema.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
import pytest
from pydantic import ValidationError

from taxi_duration.serving.schemas import PredictionRequest

def test_prediction_contract_accepts_timezone(aware_pickup: object) -> None:
    payload = PredictionRequest(
        pickup_datetime=aware_pickup,
        pickup_location_id=132,
        dropoff_location_id=230,
        passenger_count=1,
    )
    assert payload.pickup_location_id == 132

def test_prediction_contract_rejects_unknown_field(aware_pickup: object) -> None:
    with pytest.raises(ValidationError):
        PredictionRequest(
            pickup_datetime=aware_pickup,
            pickup_location_id=132,
            dropoff_location_id=230,
            passenger_count=1,
            fare_amount=60.0,
        )
```

A.59 tests/integration/test_api.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```
import json
from collections.abc import Iterator
from pathlib import Path

import joblib
import pandas as pd
import pytest
from fastapi.testclient import TestClient

from taxi_duration.config import load_config
from taxi_duration.models.artifact import runtime_versions
from taxi_duration.models.factory import build_model

VALID_PAYLOAD = {
    "pickup_datetime": "2025-01-15T08:30:00-05:00",
    "pickup_location_id": 10,
    "dropoff_location_id": 30,
    "passenger_count": 1,
}

def _write_artifact(
    directory: Path, trips: pd.DataFrame, versions: dict[str, str]
) -> tuple[Path, Path]:
    target = (
        trips["tpep_dropoff_datetime"] - trips["tpep_pickup_datetime"]
    ).dt.total_seconds() / 60
    model = build_model(load_config("configs/development.yaml"))
    model.fit(trips, target)
    model_path = directory / "model.joblib"
    metadata_path = directory / "metadata.json"
    joblib.dump(model, model_path)
    metadata_path.write_text(
        json.dumps(
            {
                "model_version": "test-v1",
                "feature_contract_version": "1.1.0",
                "runtime_versions": versions,
            }
        )
    )
    return model_path, metadata_path

def _client(monkeypatch: pytest.MonkeyPatch, model_path: Path, metadata_path: Path) -> TestClient:
    monkeypatch.setenv("TAXI_MODEL_PATH", str(model_path))
    monkeypatch.setenv("TAXI_METADATA_PATH", str(metadata_path))
    monkeypatch.setenv("TAXI_CONFIG_PATH", "configs/development.yaml")
    from taxi_duration.serving.app import app

    return TestClient(app)

@pytest.fixture()
def client(
    tmp_path: Path, synthetic_trips: pd.DataFrame, monkeypatch: pytest.MonkeyPatch
) -> Iterator[TestClient]:
```

```

paths = _write_artifact(tmp_path, synthetic_trips, runtime_versions())
with _client(monkeypatch, *paths) as test_client:
    yield test_client

def _counter(client: TestClient, status: str) -> float:
    for line in client.get("/metrics").text.splitlines():
        if line.startswith(f'taxi_prediction_requests_total{{{status}}}'):
            return float(line.rsplit(" ", 1)[1])
    return 0.0

@pytest.mark.integration
def test_prediction_endpoint(client: TestClient) -> None:
    response = client.post("/v1/predict", json=VALID_PAYLOAD)
    assert response.status_code == 200
    assert 1 <= response.json()["predicted_duration_minutes"] <= 120
    assert response.json()["feature_contract_version"] == "1.1.0"
    assert client.get("/health/ready").status_code == 200

@pytest.mark.integration
def test_contract_violations_are_counted(client: TestClient) -> None:
    before = _counter(client, "invalid")
    naive_time = {**VALID_PAYLOAD, "pickup_datetime": "2025-01-15T08:30:00"}
    assert client.post("/v1/predict", json=naive_time).status_code == 422
    assert _counter(client, "invalid") == before + 1

@pytest.mark.integration
@pytest.mark.parametrize("zone", [264, 265])
def test_non_routable_zones_are_rejected(client: TestClient, zone: int) -> None:
    payload = {**VALID_PAYLOAD, "dropoff_location_id": zone}
    assert client.post("/v1/predict", json=payload).status_code == 422

@pytest.mark.integration
@pytest.mark.parametrize("library", ["scikit-learn", "numpy", "pandas", "joblib"])
def test_runtime_mismatch_keeps_service_unready(
    tmp_path: Path,
    synthetic_trips: pd.DataFrame,
    monkeypatch: pytest.MonkeyPatch,
    library: str,
) -> None:
    versions = {**runtime_versions(), "library": "0.0.0"}
    paths = _write_artifact(tmp_path, synthetic_trips, versions)
    with _client(monkeypatch, *paths) as test_client:
        assert test_client.get("/health/live").status_code == 200
        assert test_client.get("/health/ready").status_code == 503
        assert test_client.post("/v1/predict", json=VALID_PAYLOAD).status_code == 503

```

A.60 tests/integration/test_training_pipeline.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import json
from pathlib import Path

import pandas as pd
import pytest

from taxi_duration.config import AppConfig, load_config
from taxi_duration.models.artifact import load_artifact
from taxi_duration.training.pipeline import evaluate_artifact, train_from_parquet

def _config() -> AppConfig:
    config = load_config(Path("configs/development.yaml"))
    config.gates.min_improvement_vs_baseline = 0.0
    config.gates.max_validation_test_gap = 20.0
    return config

def _train(
    tmp_path: Path, data_path: Path | list[Path], name: str, **kwargs: object
) -> dict[str, object]:
    return train_from_parquet(
        data_path,
        _config(),
        Path("configs/development.yaml"),
        tmp_path / name / "model.joblib",
        tmp_path / name / "metadata.json",
        tmp_path / name / "metrics.json",
        f"{name}-v1",
        reference_path=tmp_path / name / "reference_profile.json",
        **kwargs, # type: ignore[arg-type]
    )

@pytest.mark.integration
def test_training_produces_loadable_artifacts(
    tmp_path: Path, synthetic_trips: pd.DataFrame

```

```

) -> None:
    data_path = tmp_path / "trips.parquet"
    synthetic_trips.to_parquet(data_path)
    metrics = _train(tmp_path, data_path, "candidate")

    _, metadata = load_artifact(
        tmp_path / "candidate" / "model.joblib", tmp_path / "candidate" / "metadata.json"
    )
    assert metadata["runtime_versions"]["scikit-learn"]
    assert metadata["feature_contract_version"] == "1.2.0"
    assert (tmp_path / "candidate" / "reference_profile.json").exists()
    model_info = metrics["model"]
    assert isinstance(model_info, dict)
    assert 1 <= model_info["selected_iterations"] <= model_info["max_iterations"]
    champion = metrics["champion"]
    assert isinstance(champion, dict)
    assert champion["status"] == "skipped"
    assert champion["detail"] == "no champion supplied"
    breakdown = metrics["test_breakdown"]
    assert isinstance(breakdown, dict)
    assert "by_route_coverage" in breakdown

@pytest.mark.integration
def test_champion_is_scored_on_candidate_test_window(
    tmp_path: Path, synthetic_trips: pd.DataFrame
) -> None:
    data_path = tmp_path / "trips.parquet"
    synthetic_trips.to_parquet(data_path)
    _train(tmp_path, data_path, "champion")
    metrics = _train(
        tmp_path,
        data_path,
        "candidate",
        champion_model_path=tmp_path / "champion" / "model.joblib",
        champion_metadata_path=tmp_path / "champion" / "metadata.json",
    )
    champion = metrics["champion"]
    assert isinstance(champion, dict)
    assert champion["status"] == "compared"
    assert champion["champion_version"] == "champion-v1"
    # Identical data and seed -> identical predictions: zero delta, inside any positive margin.
    assert champion["mae_delta"] == 0.0
    assert champion["clusters"] > 1
    assert champion["noninferiority_margin_minutes"] > 0
    assert metrics["release_gate"]["passed"]

@pytest.mark.integration
def test_out_of_period_evaluation(tmp_path: Path, synthetic_trips: pd.DataFrame) -> None:
    data_path = tmp_path / "trips.parquet"
    synthetic_trips.to_parquet(data_path)
    _train(tmp_path, data_path, "candidate")
    later = synthetic_trips.copy()
    shift = pd.Timedelta(days=31)
    later["tpep_pickup_datetime"] += shift
    later["tpep_dropoff_datetime"] += shift
    later_path = tmp_path / "later.parquet"
    later.to_parquet(later_path)

    report = evaluate_artifact(
        later_path,
        _config(),
        tmp_path / "candidate" / "model.joblib",
        tmp_path / "candidate" / "metadata.json",
    )
    assert report["metrics"]["mae_minutes"] > 0
    assert set(report["breakdown"]) >= {"by_pickup_hour", "by_weekday", "by_route_coverage"}
    json.dumps(report) # report must be serialisable

@pytest.mark.integration
def test_rolling_window_records_all_source_periods(
    tmp_path: Path, synthetic_trips: pd.DataFrame
) -> None:
    paths: list[Path] = []
    for offset, period in enumerate(("2025-01", "2025-02", "2025-03")):
        frame = synthetic_trips.copy()
        shift = pd.DateOffset(months=offset)
        frame["tpep_pickup_datetime"] += shift
        frame["tpep_dropoff_datetime"] += shift
        path = tmp_path / f"yellow_tripdata_{period}.parquet"
        frame.to_parquet(path)
        paths.append(path)
    metrics = _train(tmp_path, paths, "rolling")
    assert metrics["source_periods"] == ["2025-01", "2025-02", "2025-03"]
    assert metrics["data_quality"]["valid_rows"] == 3 * len(synthetic_trips)

```

A.61 tests/unit/test_config.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from pathlib import Path

from taxi_duration.config import canonical_config_json, load_config

def test_inherited_config_overrides_base() -> None:
    config = load_config(Path("configs/development.yaml"))
    assert config.environment == "development"
    assert config.model.max_iter == 80
    assert config.data.valid_zone_max == 263

def test_canonical_config_includes_resolved_base_values(tmp_path: Path) -> None:
    base = tmp_path / "base.yaml"
    child = tmp_path / "child.yaml"
    base.write_text(Path("configs/base.yaml").read_text(encoding="utf-8"), encoding="utf-8")
    child.write_text("extends: base.yaml\nenvironment: test\n", encoding="utf-8")
    before = canonical_config_json(load_config(child))
    base.write_text(
        base.read_text(encoding="utf-8").replace("random_seed: 42", "random_seed: 43"),
        encoding="utf-8",
    )
    after = canonical_config_json(load_config(child))
    assert before != after

```

A.62 tests/unit/test_drift.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from pathlib import Path

import numpy as np
import pandas as pd

from taxi_duration.config import load_config
from taxi_duration.monitoring.drift import drift_report
from taxi_duration.monitoring.profile import build_reference_profile

def test_invalid_values_are_not_hidden_by_renormalisation(
    tmp_path: Path, synthetic_trips: pd.DataFrame
) -> None:
    reference = tmp_path / "reference.json"
    build_reference_profile(
        synthetic_trips.assign(
            duration_minutes=(
                synthetic_trips["tpep_dropoff_datetime"] - synthetic_trips["tpep_pickup_datetime"]
            ).dt.total_seconds()
            / 60
        ),
        reference,
    )
    config = load_config("configs/development.yaml").monitoring
    clean_report = drift_report(synthetic_trips, reference, config)
    assert clean_report["status"] == "ok"
    assert set(clean_report["categorical"]) == {"pickup_zone", "dropoff_zone", "route"}
    assert clean_report["categorical"]["route"]["unseen_rate"] == 0.0

    corrupted = synthetic_trips.copy()
    corrupted["passenger_count"] = corrupted["passenger_count"].astype(float)
    corrupted.loc[:23, "passenger_count"] = np.nan # 10%
    corrupted.loc[24:47, "passenger_count"] = 0 # 10%
    report = drift_report(corrupted, reference, config)

    passenger = report["features"]["passenger_count"]
    assert passenger["data_quality"]["missing_rows"] == 24
    assert passenger["data_quality"]["out_of_range_rows"] == 24
    assert passenger["data_quality"]["invalid_rate"] == 0.2
    assert passenger["status"] == "critical"
    assert report["status"] == "critical"

```

A.63 tests/unit/test_features.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import numpy as np
import pandas as pd

from taxi_duration.features.builder import FEATURE_COLUMNS, TripFeatureBuilder

def test_features_are_finite_and_leakage_safe(synthetic_trips: pd.DataFrame) -> None:
    transformed = TripFeatureBuilder().fit_transform(synthetic_trips)
    assert tuple(transformed.columns) == FEATURE_COLUMNS
    assert "tpep_dropoff_datetime" not in transformed.columns
    assert "duration_minutes" not in transformed.columns
    assert np.isfinite(transformed.to_numpy()).all()

def test_cyclical_hour_wraps_smoothly() -> None:
    frame = pd.DataFrame(

```

```

        {
            "tpep_pickup_datetime": pd.to_datetime(["2025-01-01 00:00", "2025-01-01 12:00"]),
            "PULocationID": [1, 1],
            "DOLocationID": [2, 2],
            "passenger_count": [1, 1],
        }
    )
    result = TripFeatureBuilder().fit_transform(frame)
    assert np.isclose(result.loc[0, "hour_cos"], 1.0)
    assert np.isclose(result.loc[1, "hour_cos"], -1.0)

```

A.64 tests/unit/test_gates.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import numpy as np
import pytest

from taxi_duration.config import load_config
from taxi_duration.training.gates import (
    ChampionComparison,
    evaluate_release_gates,
    paired_delta_upper_bound,
)

GOOD_VALIDATION = {"mae_minutes": 4.0}
GOOD_TEST = {"mae_minutes": 4.5, "p95_absolute_error_minutes": 15.0}

def test_release_gate_passes_good_candidate() -> None:
    config = load_config("configs/development.yaml").gates
    result = evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config)
    assert result.passed

def test_release_gate_explains_failure() -> None:
    config = load_config("configs/development.yaml").gates
    result = evaluate_release_gates(
        {"mae_minutes": 14.0},
        {"mae_minutes": 20.0, "p95_absolute_error_minutes": 40.0},
        baseline_validation_mae=14.1,
        config=config,
    )
    assert not result.passed
    assert len(result.failures) >= 2

def _compared(champion_mae: float, upper: float) -> ChampionComparison:
    return ChampionComparison(
        status="compared", champion_test_mae=champion_mae, mae_delta_upper_confidence=upper
    )

def test_candidate_not_shown_non_inferior_is_blocked() -> None:
    config = load_config("configs/development.yaml").gates # margin 1% of champion MAE
    # champion MAE 5.0 -> margin 0.05 min; upper bound 0.08 exceeds it.
    result = evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, _compared(5.0, 0.08))
    assert not result.passed
    assert "non-inferior" in result.failures[0]

def test_equal_candidate_within_margin_passes() -> None:
    config = load_config("configs/development.yaml").gates
    # Upper bound slightly above zero (no evidence of improvement) but inside the margin.
    assert evaluate_release_gates(
        GOOD_VALIDATION, GOOD_TEST, 7.0, config, _compared(5.0, 0.03)
    ).passed

def test_incomplete_comparison_blocks_release() -> None:
    config = load_config("configs/development.yaml").gates
    champion = ChampionComparison(status="compared", champion_test_mae=5.0)
    assert not evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, champion).passed

def test_zero_margin_is_rejected_by_config() -> None:
    from pydantic import ValidationError

    from taxi_duration.config import GateConfig

    payload = load_config("configs/development.yaml").gates.model_dump()
    payload["max_regression_vs_champion"] = 0.0
    with pytest.raises(ValidationError):
        GateConfig.model_validate(payload)

def test_equally_good_candidates_mostly_pass() -> None:
    """Regression test for the v1.2 superiority behaviour (~95% of equal candidates blocked)."""
    config = load_config("configs/development.yaml").gates
    rng = np.random.default_rng(0)
    champion_mae, n_rows, trials = 7.0, 20_000, 200
    clusters = np.arange(n_rows) // 60 # one-hour blocks of one-minute rows

```

```

passed = superior = 0
for _ in range(trials):
    hour_effect = np.repeat(rng.normal(0.0, 0.3, n_rows // 60 + 1), 60)[:n_rows]
    delta = hour_effect + rng.normal(0.0, 3.0, n_rows) # true mean difference = 0
    stat = paired_delta_upper_bound(delta, clusters, config.champion_confidence_level)
    champion = _compared(champion_mae, stat.upper_confidence)
    passed += evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, champion).passed
    superior += stat.upper_confidence <= 0.0 # what a zero-margin gate would require
# Expected pass rate here is ~0.83 (margin 0.07 min vs cluster-robust SE ~0.027 min); a
# zero-margin (superiority) gate would pass only ~5%. The pass rate rises with test volume.
assert passed / trials > 0.7
assert superior / trials < 0.15

def test_clearly_worse_candidates_are_blocked() -> None:
    config = load_config("configs/development.yaml").gates
    rng = np.random.default_rng(1)
    n_rows = 20_000
    clusters = np.arange(n_rows) // 60
    delta = 0.5 + rng.normal(0.0, 3.0, n_rows) # candidate 0.5 min worse per trip
    stat = paired_delta_upper_bound(delta, clusters, config.champion_confidence_level)
    champion = _compared(7.0, stat.upper_confidence)
    assert not evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, champion).passed

def test_cluster_robust_error_exceeds_iid_error_under_hour_correlation() -> None:
    rng = np.random.default_rng(2)
    n_rows = 12_000
    clusters = np.arange(n_rows) // 60
    delta = np.repeat(rng.normal(0.0, 1.0, n_rows // 60), 60) + rng.normal(0.0, 1.0, n_rows)
    clustered = paired_delta_upper_bound(delta, clusters, 0.95).standard_error
    iid = paired_delta_upper_bound(delta, np.arange(n_rows), 0.95).standard_error
    assert clustered > 3 * iid

def test_statistic_requires_two_clusters() -> None:
    with pytest.raises(ValueError, match="two clusters"):
        paired_delta_upper_bound(np.array([0.1, -0.2, 0.3]), np.zeros(3), 0.95)

def test_unevaluable_champion_blocks_release() -> None:
    config = load_config("configs/development.yaml").gates
    champion = ChampionComparison(status="failed", detail="incompatible runtime")
    assert not evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, champion).passed

def test_missing_champion_is_bootstrap_not_failure() -> None:
    config = load_config("configs/development.yaml").gates
    champion = ChampionComparison(status="skipped")
    assert evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, champion).passed

def test_data_quality_failure_blocks_release() -> None:
    config = load_config("configs/development.yaml").gates
    quality = {
        "rejection_rate": 0.20,
        "out_of_period_rate": 0.0,
        "duplicate_rate": 0.0,
        "valid_rows": 1_000,
    }
    result = evaluate_release_gates(GOOD_VALIDATION, GOOD_TEST, 7.0, config, data_quality=quality)
    assert not result.passed
    assert "data rejection rate exceeds limit" in result.failures

```

A.65 tests/unit/test_ingest.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import httpx
import pytest

from taxi_duration.data import ingest
from taxi_duration.data.validation import Period

def test_latest_available_period_falls_back(monkeypatch: pytest.MonkeyPatch) -> None:
    available = {Period(2026, 5)}

    def fake_available(period: Period, _: str, timeout_seconds: float = 20.0) -> bool:
        del timeout_seconds
        return period in available

    monkeypatch.setattr(ingest, "source_available", fake_available)
    resolved = ingest.latest_available_period(Period(2026, 7), "ignored", 6)
    assert resolved == Period(2026, 5)

def test_latest_available_period_fails_after_bound(monkeypatch: pytest.MonkeyPatch) -> None:
    def never_available(period: Period, template: str, timeout_seconds: float = 20.0) -> bool:
        del period, template, timeout_seconds
        return False

```

```

monkeypatch.setattr(ingest, "source_available", never_available)
try:
    ingest.latest_available_period(Period(2026, 7), "ignored", 2)
except RuntimeError as error:
    assert "no published TLC source" in str(error)
else:
    raise AssertionError("bounded source lookup should fail explicitly")

@pytest.mark.parametrize(
    ("status", "expected"), [(200, True), (206, True), (403, False), (404, False)]
)
def test_source_status_mapping(
    monkeypatch: pytest.MonkeyPatch, status: int, expected: bool
) -> None:
    monkeypatch.setattr(ingest, "_probe_status", lambda url, timeout: status)
    assert ingest.source_available(Period(2026, 5), "{year}-{month}", attempts=1) is expected

def test_network_error_is_not_treated_as_unpublished(monkeypatch: pytest.MonkeyPatch) -> None:
    def broken(url: str, timeout: float) -> int:
        raise httpx.ConnectError("connection reset")

    monkeypatch.setattr(ingest, "_probe_status", broken)
    with pytest.raises(ingest.SourceProbeError, match="ConnectError"):
        ingest.source_available(Period(2026, 5), "{year}-{month}", backoff_seconds=0.0)

def test_transient_error_is_retried(monkeypatch: pytest.MonkeyPatch) -> None:
    responses = iter([503, 200])
    monkeypatch.setattr(ingest, "_probe_status", lambda url, timeout: next(responses))
    assert ingest.source_available(Period(2026, 5), "{year}-{month}", backoff_seconds=0.0)

def test_probe_error_stops_the_lookback(monkeypatch: pytest.MonkeyPatch) -> None:
    def outage(period: Period, template: str, timeout_seconds: float = 20.0) -> bool:
        raise ingest.SourceProbeError("outage")

    monkeypatch.setattr(ingest, "source_available", outage)
    with pytest.raises(ingest.SourceProbeError):
        ingest.latest_available_period(Period(2026, 7), "ignored", 6)

```

A.66 tests/unit/test_monitoring.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import pytest

from taxi_duration.monitoring.profile import population_stability_index

def test_psi_is_zero_for_identical_distributions() -> None:
    assert population_stability_index([0.2, 0.8], [0.2, 0.8]) == pytest.approx(0.0)

def test_psi_detects_shift() -> None:
    assert population_stability_index([0.2, 0.8], [0.8, 0.2]) > 1.0

```

A.67 tests/unit/test_period.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

from datetime import date
from pathlib import Path

import pandas as pd
import pytest

from taxi_duration.cli import default_period, period_window
from taxi_duration.config import load_config
from taxi_duration.data.validation import Period, validate_and_clean
from taxi_duration.training.split import chronological_split

@pytest.mark.parametrize("text", ["2025-1", "2025-01"])
def test_period_parse_normalises_month(text: str) -> None:
    assert str(Period.parse(text)) == "2025-01"

def test_period_from_tlc_filename() -> None:
    assert Period.from_filename(Path("yellow_tripdata_2025-03.parquet")) == Period(2025, 3)
    assert Period.from_filename(Path("sample.parquet")) is None

@pytest.mark.parametrize(
    ("today", "expected"),
    [(date(2026, 9, 15), "2026-07"), (date(2026, 1, 15), "2025-11"), (date(2026, 2, 1), "2025-12")],
)
def test_default_period_lags_two_months(today: date, expected: str) -> None:
    assert str(default_period(today)) == expected

```

```

def test_period_window_crosses_year_boundary() -> None:
    assert [str(period) for period in period_window(Period(2025, 1), 3)] == [
        "2024-11",
        "2024-12",
        "2025-01",
    ]

def test_out_of_period_rows_are_removed_and_reported(synthetic_trips: pd.DataFrame) -> None:
    stray = synthetic_trips.head(3).copy()
    stray["tpep_pickup_datetime"] = pd.to_datetime(
        ["2008-12-31 23:00", "2009-01-01 00:10", "2025-02-01 00:05"]
    )
    stray["tpep_dropoff_datetime"] = stray["tpep_pickup_datetime"] + pd.Timedelta(minutes=12)
    frame = pd.concat([synthetic_trips, stray], ignore_index=True)
    config = load_config("configs/development.yaml")

    clean, report = validate_and_clean(frame, config.data, Period(2025, 1))

    assert report.out_of_period_rows == 3
    assert len(clean) == len(synthetic_trips)
    split = chronological_split(clean, config.split)
    assert split.train["tpep_pickup_datetime"].min() >= pd.Timestamp("2025-01-01")
    assert split.test["tpep_pickup_datetime"].max() < pd.Timestamp("2025-02-01")

def test_non_routable_zones_are_rejected_in_training(synthetic_trips: pd.DataFrame) -> None:
    frame = synthetic_trips.copy()
    frame.loc[4, "DOLocationID"] = 264
    frame.loc[5:9, "PULocationID"] = 265
    _, report = validate_and_clean(frame, load_config("configs/development.yaml").data)
    assert report.rejected_rows == 10

```

A.68 tests/unit/test_predict.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import numpy as np
import pandas as pd

from taxi_duration.config import load_config
from taxi_duration.models.predict import predict_duration

class ExtremeModel:
    def predict(self, features: pd.DataFrame) -> np.ndarray:
        del features
        return np.asarray([-20.0, 500.0])

def test_shared_prediction_contract_clips_both_bounds() -> None:
    serving = load_config("configs/development.yaml").serving
    prediction = predict_duration( # type: ignore[arg-type]
        ExtremeModel(), pd.DataFrame(index=range(2)), serving
    )
    assert prediction.tolist() == [serving.prediction_lower_bound, serving.prediction_upper_bound]

```

A.69 tests/unit/test_validation.py

Repository infrastructure or reference file; inline comments and surrounding chapters define its execution role.

```

import pandas as pd

from taxi_duration.config import load_config
from taxi_duration.data.validation import validate_and_clean

def test_validation_rejects_impossible_duration(synthetic_trips: pd.DataFrame) -> None:
    bad = synthetic_trips.iloc[[0]].copy()
    bad["tpep_dropoff_datetime"] = bad["tpep_pickup_datetime"]
    frame = pd.concat([synthetic_trips, bad], ignore_index=True)
    clean, report = validate_and_clean(frame, load_config("configs/development.yaml").data)
    assert len(clean) == len(synthetic_trips)
    assert report.rejected_rows == 1

def test_validation_rejects_missing_columns(synthetic_trips: pd.DataFrame) -> None:
    config = load_config("configs/development.yaml").data
    try:
        validate_and_clean(synthetic_trips.drop(columns="PULocationID"), config)
    except ValueError as error:
        assert "PULocationID" in str(error)
    else:
        raise AssertionError("missing schema column should fail")

def test_validation_drops_and_reports_exact_duplicates(synthetic_trips: pd.DataFrame) -> None:
    frame = pd.concat([synthetic_trips, synthetic_trips.iloc[:2]], ignore_index=True)
    clean, report = validate_and_clean(frame, load_config("configs/development.yaml").data)
    assert len(clean) == len(synthetic_trips)
    assert report.duplicate_rows == 2

```

```
assert report.duplicate_rate == 2 / len(frame)
```